Basic Data Types in Python

DATA 118 Lab — Student Version

Goals

The goal of this tutorial is to introduce you to the style of tutorials we will be using to learn to program and to review some of what you've learned about basic data types in Python. We will cover:

- Ints
- Floats
- Booleans
- Strings

How Does This Tutorial Work?

Before we ask you to start working in a Jupyter notebook, we will ask you to complete this tutorial in a web browser so you can get used to one thing at a time.

Below is an example of a *cell*. Type some basic Python code into a cell. Remember that Python uses built-in operators to do math calculations if you can't think of anything else to type.

Exercise — write your code here:

Now, click on the light blue "Run Code" button at the top of the cell. Hopefully, you should see some output in a separate box just below the cell.

You can type anything into the cell, but usually, you should be typing code to answer to a question we are asking. For example, write some code to answer the question "What is two plus two" in the cell below:

Exercise — write your code here:

This time, you should see a light "Submit Answer" button at the top of the cell. Click on it–what do you see? If you wrote something like `2+2`, you should see a green box saying that you got the right answer.

Now, write some code that will tell you anything BUT the answer to the question "What is two plus two."

Exercise — write your code here:

Click on the "Submit Answer" button. This time, you should see a red box saying that your answer was incorrect. Most of the time, this means that you should revisit your code–you can submit the answer multiple times until you get the right answer. For now, you can move on!

Sometimes, the cells expect your code to match ours exactly. For example, edit the code in the cell below to give the answer to $2 + 2 - 3 + 3$ and click on the "Submit Answer" button.

Exercise

```
2+2
```

Even though `2+2` and `2+2-3+3` give the same value, `4`, the tutorial is telling you that you didn't get the right answer because your code is not an exact match. In this class, we will usually use exact code matching so that you can learn the basics, but there might be times where there are multiple ways to get the right answer. You are always welcome to experiment and ask us for help!

Sometimes, you might see a yellow box with a strange error. If you do, please know that you are likely not making a mistake and we need to fix the issue on the back end. If you see something like this, please let your instructor know!

Syntax Highlighting

Remember that syntax is "the arrangement of words and phrases to create well-formed sentences in a language", a.k.a., coding grammar. Syntax highlighting displays code in different colors and fonts to help programmers identify different pieces of code quickly.

The syntax highlighting on my slides is different from the syntax highlighting in this tutorial, which may be different from the syntax highlighting you are using. Let's take a quick look at the syntax highlighting in your tutorial.

Exercise

```
2+2
"Hello, my name is Professor Nussbaum."
7.7
True
```

Question 1. What colors do you see? Check all that apply:
☐ Black
☐ Dark Blue
☐ Gray
☐ Green
☐ Purple
☐ Red
☐ Light Blue
☐ Yellow

Our 4 lines of code are not colored differently just for fun–the colors are meant to help us identify something.

Question 2. What do you think the colors are meant to signify? Check all that apply:
☐ Longer inputs are green.
☐ Numbers are dark blue.
☐ All capitalized inputs are light blue.
☐ Strings are green.
☐ Gray is used for errors.
☐ True and False are light blue.

Discuss with a neighbor (or on Ed):
Open an empty Jupyter notebook and check to see what colors `ints`, `floats`, strings and Booleans are on your own computer. Is it the same color as your neighbor?

Ints and Floats

Remember that numbers can be stored as `ints` or `floats` in Python. An `int` is a number without a decimal point; a `float` usually has a decimal point (or is stored in scientific notation). Remember also that the `type()` function will tell us what kind of object we are working with.

Edit the code below to find the types of the numbers:

Exercise

```
18
22.7
```

Question 3. What is the type of `18`?

☐ `Boolean`
☐ `int`
☐ `float`
☐ `str`, the abbreviation for string

Question 4. What is the type of `22.7`?

☐ `Boolean`
☐ `int`
☐ `float`
☐ `str`, the abbreviation for string

Other types have more specific methods, but for ints and floats, we are happy using simple operators or comparison operators.

Operation	Operator	Example	Output
Addition	`+`	`2+3`	`5`
Subtraction	`-`	`2-3`	`-1`
Multiplication	`*`	`2*3`	`6`
Division	`/`	`7/3`	`2.33`
Floor Division	`//`	`7//3`	`2`
Remainder (mod)	`%`	`7%3`	`1`
Exponentiation	`**`	`2**0.5`	`1.41421`

Use the cell below to experiment with the operators. Try to use the operators in sequence, with and without parentheses.

Exercise — write your code here:

Question 5. Does Python follow PEMDAS for the order of operations?
☐ Yes
☐ No

Question 6. What is the output (including the type) of `6/2`?
☐ `3`, which is an `int`
☐ `3.0`, which is a `float`

Don't forget, there are also the comparison operators:

Comparison	Operator	True Example	False Example
Less than	`<`	`2 < 3`	`2 < 2`
Greater than	`>`	`3 > 2`	`3 > 3`
Less than or equal	`<=`	`2 <= 2`	`3 <= 2`
Greater than or equal	`>=`	`3 >= 3`	`2 >= 3`
Exactly equal	`==`	`3 == 3`	`3 == 2`
Not Equal	`!=`	`3 != 2`	`3 != 3`

Use the cell below to experiment with the comparison operators. Using them with `ints` and `floats` should be familiar, but you can also use them with strings and Booleans.

Exercise — write your code here:

Question 7. What does `"Hello" < "hello"` return?
☐ `True`
☐ `False`

We can use multiple operators, as well as the parentheses, to carry out more advanced calculations. For example, some of you may be familiar with the average or mean:

$$\bar{x} = \frac{x_1 + x_2 + ... + x_n}{n}$$

In other words, to find the average of a variable, you add all the numbers up and divide by the number of observations.

Consider the heights in centimeters of the following Star Wars characters:

- Luke Skywalker–172cm
- C-3PO–167cm
- R2-D2–96cm
- Leia Organa–150cm
- Han Solo–180cm, and
- Chewbacca–238cm

Using the `+` and `/` operators, as well as the parentheses, write a line of code to calculate the average.

Exercise — write your code here:

As we have said, there are not many `int`- and `float`-specific methods, but there are some built-in functions in Python that make working with numbers easier:

Function	Description
`abs()`	Returns the absolute value of a number
`float()`	Returns (or converts an object to) a floating point number
`int()`	Returns (or converts an object to) an integer number
`round()`	Rounds a number

Convert the integer 172 to a float using one of these functions in the cell below:

Exercise

```
172
```

Question 8. Take a closer look at your code. What color is used to highlight a built-in function?

☐ Black
☐ Blue
☐ Gray
☐ Green
☐ Purple
☐ Red
☐ Yellow

The table above is a shortened version of a much larger table with built-in functions.

Discuss with a neighbor (or on Ed):
Click on the link to the larger table. Can you spot any other functions that might be useful?

Variable Assignment

Let's revisit the heights of the Star Wars characters. Consider Chewbacca in particular–it turns out that his height is 228cm, not 238. If we were correcting a data analysis, we would have to re-type in all those numbers to find the correct mean. However, if you were handing it off to a colleague or if enough time had passed, you may not remember exactly why you wrote what you wrote.

Sometimes, to make it easier to track our work, we use something called *variable assignment*. Variable assignment helps us store values to work with using a variable name and the *assignment operator*, `=`.

Consider the following code:

Exercise

```
Chewbacca = 238
```

In this case, the variable name is `Chewbacca`. Notice that it appears in black! Now, edit the code above so that `Chewbacca` is equal to 228, not 238.

This time, if you click on the "Run Code" button, it looks like nothing happens. That's not technically true–we corrected the height by re-storing the value, but we can't necessarily see what happened. Another built-in function is the `print()` function. Click on the "Run Code" button below, and then edit Chewbacca's height–does the output change at all?

Exercise

```
Chewbacca = 238
print(Chewbacca)
```

Now, add some code to assign heights to the rest of our characters, Luke Skywalker, C-3PO, R2-D2, Leia Organa, and Han Solo. Remember, the heights are:

- Luke Skywalker–172cm
- C-3PO–167cm
- R2-D2–96cm
- Leia Organa–150cm
- Han Solo–180cm, and
- Chewbacca–228cm

Exercise

```
Chewbacca = 228
Luke = ...
C3P0 = ...
R2D2 = ...
Leia = ...
Han = ...
```

Variable names *can* be anything, but it's generally good to avoid the names of the built-in functions as well as the Python keywords, listed below:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Note that the keywords ALSO appear as blue.

Booleans

Now let's practice working with Booleans using our variables, `Chewbacca`, `Luke`, `C3P0`, `R2D2`, `Leia`, and `Han`. Remember that Booleans are outputted by the comparison operators:

Comparison	Operator	True Example	False Example
Less than	<	2 < 3	2<2
Greater than	>	3 > 2	3>3
Less than or equal	<=	2 <=2	3<=2
Greater than or equal	>=	3 >= 3	2 >= 3
Exactly equal	==	3 == 3	3 == 2
Not Equal	!=	3 != 2	3 != 3

Is Luke taller than Leia? Write a line of code using the right comparison operator and the variable names. This is a good time to note that our tutorial remembers the variables we assigned earlier!

Exercise — write your code here:

Is Chewbacca shorter than Han Solo? Write a line of code using the right comparison operator and the variable names.

Exercise — write your code here:

It can be helpful to combine some of these comparisons using Python's logical operators. Click on the link and read about these new operators. Then, use them to complete the questions below. You may also find it helpful to use parentheses.

Is Leia taller than *either* of the droids, R2-D2 or C-3PO?

Exercise — write your code here:

Is Chewbacca taller than *both* of the droids, R2-D2 or C-3PO?

Exercise — write your code here:

Strings

Saving the most complicated for last! Unlike `ints`, `floats`, and Booleans, strings have LOTS of methods we can use. Remember that unlike a function, a method is a tool associated with specific type of object.

First, let's create a variable for Leia Organa's full name:

Exercise

```
Leia_fullname = "Leia Organa"
```

A few things:

1. First, we are choosing separate variable name for Leia's full name–if we saved it as `Leia`, we would lose the information about her height. Be careful in your own work not to overwrite variables unless you really, really mean it!
2. `Leia_fullname` is the name of a variable, whereas `"Leia Organa"` is a string stored as `Leia_fullname`. Remember that strings begin and end with quotation marks, and show up as green, whereas variable names are black.

Strings are the same whether you use double or single quotes to define them. Write a line of code to see if `Leia_fullname` is the same as `'Leia Organa'` using a comparison operator.

Exercise — write your code here:

Make sure you are careful to use the single equals sign, `=`, when assigning variables, and the double equals sign, `==`, when checking equality!

Now, write a line of code using a comparison operator to see if `Leia_fullname` is equal to `"leia organa"`.

Exercise — write your code here:

Python does not consider two strings to be the same unless they match exactly. This is where some of the methods come in. The `.lower()` method should convert all of the characters in a string to lowercase. Try and use it with our variable name in the cell below. Save the lowercase version as a new variable, `Leia_lower`, and check to see if it is equal to `"leia organa"`.

Exercise

```
leia_lower = Leia_fullname...
...
```

Now, use the `.capitalize()` method to try and undo what you just did.

Discuss with a neighbor (or on Ed):
Will `.capitalize()` undo what you did completely? If not, what function can undo it?

Exercise

```
leia_lower...
...
```

Like I said, there are many many string methods. Check out a table of string methods–can you find one that will do the opposite of `.lower()`?

Question 9. What is the method that will capitalize all of the letters in a string?
☐ `.isnumeric()`
☐ `.replace()`
☐ `.split()`
☐ `.strip()`
☐ `.upper()`

So far, we have only considered fairly short strings, but they can be relatively long, as well! Consider Princess Leia's message to Obi-Wan Kenobi in *A New Hope*.

Exercise

```
message = "General Kenobi. Years ago you served my father in the Clone
 ↪  Wars. Now he begs you to help him in his struggle against the
 ↪  Empire. I regret that I am unable to present my father's request
 ↪  to you in person, but my ship has fallen under attack, and I'm
 ↪  afraid my mission to bring you to Alderaan has failed. I have
 ↪  placed information vital to the survival of the Rebellion into the
 ↪  memory systems of this R2 unit. My father will know how to
 ↪  retrieve it. You must see this droid safely delivered to him on
 ↪  Alderaan. This is our most desperate hour. Help me, Obi-Wan
 ↪  Kenobi. You're my only hope."
```

We might be interested in locating shorter strings within larger strings. For example, we could examine the contents of `message` to find how many times the shorter string `"Kenobi"` appears, but the longer the string, the more difficult this task will be. Instead, let's consider the `.count()` method. This method is different than many we have seen, because we have to pass what is known as an "argument". An argument is a value that you pass into a function or method–in this case, the argument is the shorter string, `"Kenobi"`.

We can change the argument. Edit the code below so that we count the number of

times the string `"father"` occurs in `message`.

Exercise

```
message.count("Kenobi")
```

Discuss with a neighbor (or on Ed):
Go back to the string methods table and find an interesting method. See if you can use it with `message`.

Exercise — write your code here:

In addition to having specific methods, some of the operators we are already familiar with operate differently with strings. Consider the following cells. What happens if you add the two strings together?

Exercise

```
obiwan = "These aren't the droids you're looking for."
stormtrooper = "These aren't the droids we're looking for."
...
```

When used with strings, the `+` is the concatenation operator. It pastes the two messages together, exactly as they are–notice that there is no space in between the sentences in the concatenated message, even though we would normally put one there.

What happens if you use the multiplication operator, `*`, with `message_short`? Multiply it by 3.

Exercise

```
message_short = "Help me, Obi-Wan Kenobi. You're my only hope."
...
```

You should notice that the multiplication operator will repeat the string (in this case, three times). What happens if you multiply by 1.5 instead?

Exercise

```
message_short = "Help me, Obi-Wan Kenobi. You're my only hope."
...
```

You should get an error! Remember that sometimes error messages are frustrating, but they are supposed to help us debug our code. In this case, we got a `TypeError`, because we tried to multiply a string (allowed) by an object of the `float` type (not allowed). The multiplication operator only works with type `int`.

Python Data Storage

DATA 118 Lab — Student Version

Goals

The goal of this lab is review some of what you've learned about coding in Python. We will cover:

- Lists
- Sets
- Tuples
- Arrays
- Dictionaries

Storing Data in Python

Python and its libraries provide many ways to group data together. Some important ones:

- Lists, Tuples, Sets, Dictionaries (built-in to Python)
- Arrays (found in the `numpy` library)
- DataFrames (in `pandas`)

The above are listed in order of increasing functionality and sophistication. In general you should use the simplest one that meets your needs.

Lists

A **list** is an ordered sequence of values that can be changed.

You can create a list by using square brackets "`[]`". Each element is separated by commas.

Let's create a list!

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]
School_locations
```

Lists can have different data types!

Exercise

```
Random_stuff = ["California", 38, 3.14159] #A string, int, float
Random_stuff
```

Built-in Functions Recall Python has built-in functions of the form `function_name()` A very common one is the `print()` function!

`type()` is another important function. We can call this on a number, string, list, or any object to see type it is!

Exercise

```
print("California")
print(type("California"))
print(type(38))
print(type("38"))
print(type(3.14))
print(type(Random_stuff))
```

We can also find the length of a list:

Exercise

```
len(Random_stuff)
```

Accessing values from a list Each element of a sequence is assigned an index corresponding to its position where **indices start at 0**. We can access an element by calling the sequence or list and putting in square brackets the element we want!

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
↪  "Texas", "Georgia", "Washington D.C."]
School_locations[1]
```

Negative indexes count from the end of the sequence.

Exercise

```
School_locations[-1]
```

If our list is long or we want the first or second from end, this is useful!

1. What will be the result of running the following code? Answer in the box below.

```
School_locations = ["California", "Illinois", "North Carolina", "Texas",
 ↪  "Georgia", "Washington D.C."]

School_locations[6]
```

Question 1. What will be the result of running the code above?
☐ 'California'
☐ 'Illinois'
☐ 'North Carolina'
☐ 'Texas'
☐ 'Georgia'
☐ 'Washinton D.C.'
☐ An error

List Slicing We can also extract a "slice" of a list.

The range of elements can be specified with colons. The output is a list starting at the left index and stopping at (right index -1). The range you specify is a half-closed interval: [start,end).

```
list[start: end]
```

Exercise

```
School_locations[1:3] # up to but not including the end of the slice
```

The above prints out the second and third elements.

We can still slice a list by 'leaving out' a starting (or ending) position. The missing position will revert to the default start value: the beginning of the list. The default end value is the end of the list.

Exercise

```
School_locations[:3] #start at beginning stop at index end-1 -----> 3-1=2
```

2. What will the following output? Answer in the box below.

```
School_locations = ["California", "Illinois", "North Carolina", "Texas",
 ↪  "Georgia", "Washington D.C."]

School_locations[:]
```

Question 2. What will be the result of running the code above?
☐ `[]`
☐ `['California', 'Washington D.C.']`
☐ `['California', 'Illinois', 'North Carolina', 'Texas', 'Georgia', 'Washington D.C.']`
☐ An error

Default Settings: List Slicing It might be useful to take out every even index from a list.

There is an optional argument we can use when slicing a list, the step. In general we can slice by using `list[start: end: step]` where the default (what is used when it isn't specified) of step is 1

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]

School_locations[::2]
```

3. What do you think the following will output? Answer in the box below.

```
School_locations[:3:3]
```

Question 3. What will be the result of running the code above?
☐ `[]`
☐ `['California']`
☐ `['California', 'Texas']`
☐ `['North Carolina', 'Washington D.C.']`
☐ `['North Carolina']`
☐ `['California', 'Illinois', 'North Carolina', 'Texas', 'Georgia', 'Washington D.C.']`
☐ An error

Operations and manipulation on lists We can:

- Insert items into lists!
  - At either end
  - In the middle

- Count how many items have a specific value.
- Sort elements.

How do we do this? By using *methods*. *Methods* are particular built-in functions that work on objects in Python. There are specific methods that work for all *list* objects!

List methods take the form:

```
list.method()
```

Built-in functions can also be applied to objects (including lists) in Python. Recall that a function applied to a list will take the form:

```
function_name(list)
```

Appending to a list You can append an item to the end of a list:

Exercise

```
School_locations = ["California", "Illinois", "North Carolina",
 ↪  "Texas", "Georgia", "Washington D.C."]

School_locations.append('Michigan')
School_locations
```

Or you can insert a value at a particular index:

Exercise

```
School_locations.insert(4,'tomato')
School_locations
```

Note we didn't assign the result to `School_locations`, but this **automatically changes/overwrites** the object.

Let's make a copy of our list so we can see how this works.

Exercise

```
School_location_og = School_locations.copy()
School_location_og
```

4. Append the number 2023 to `School_locations` and then view the list. Note even though the list contains only strings, I can append an integer as lists accept any data type.

Exercise

```
...

School_locations
```

This does not change our copy of the list!

Exercise

```
School_location_og
```

How do we find methods? Use online documentation!

Or use the built-in function `dir()`.

Exercise

```
print(dir(School_locations))

print(dir(list))
```

Let's play with lists! Create a list with the following words as strings: classify, evaluate, great, dorm, inside. Name the list `my_list`.

Exercise

```
my_list
```

Remove the 2nd and 3rd elements. You may want to check the documentation for list functions and methods! Because of how we are parsing your solutions, please leave the `my_list` line as is!

Exercise

```
...

my_list
```

Append 316 to the end of the list.

Exercise — write your code here:

Insert the string "Math is fun" at index 2.

Exercise — write your code here:

Question 4. Did your above operations change your original list, or create a copy?
☐ Change Original
☐ Create a Copy

Given list `number_list = [1,2,3,4,5,6,7]`, create a new list `new_list = [6,4,2]` in 2 lines of code.

Exercise

```
number_list = [1,2,3,4,5,6,7]
```

Sets

Let's say that you want to find all unique elements in the following list:

Exercise

```
grocery_list = ['apple', 'apple', 'milk', 'cheese', 'egg', 'egg',
 ↪  'egg']
```

The best way to do this with a list is to write a loop or list comprehension. However, there is a better way to carry out tasks like this using a different data type: a **set**. A **set** is an unordered sequence of unique values that can be changed.

You can create a set by using the curly braces ({}). Each element is separated by commas. Alternatively, you can use the built-in function `set()` to convert a list to a set.

Let's create a set! First, let's try using the curly braces:

Exercise

```
grocery_set = {'apple', 'apple', 'milk', 'cheese', 'egg', 'egg', 'egg'}
grocery_set
```

Now, let's see what using `set()` looks like:

Exercise

```
grocery_set = set(grocery_list)
grocery_set
```

Count how many times each item appears in `grocery_list`, and then how many times each item appears in `grocery_set`. Notice that the duplicate values are removed, and you are left with only unique values. This is one of the advantages of sets!

Properties of Sets Sets borrow mathematical concepts as a way to maintain information on different things and possibly counts.

`in` One of the most helpful operators you can use on sets are logical operators, like `in`, which will tell you if an item is included in the set:

Exercise

```
'cheese' in grocery_set
'bread' in grocery_set
```

Intersect You can also find all of the elements that two sets have in common. This is known as finding the intersection of two sets, and uses the &:

Exercise

```
fruits_set = {'apple', 'grape', 'kiwi', 'pear', 'strawberry'}
fruits_set & grocery_set
```

Question 5. If we create a third set, `dairy_set = {'milk', 'cheese', 'cream'}`, what will the intersection of `groceries_set` and `dairy_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Union You can also find all of the elements that two sets have in total, even if the item only belongs to one set. This is known as finding the union of two sets, and uses the |:

Exercise

```
fruits_set | grocery_set
```

Question 6. Again, let `dairy_set = {'milk', 'cheese', 'cream'}`. What will the union of `groceries_set` and `dairy_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Difference Finally, you can find all of the elements in one set, but not the other. This is known as finding the difference between two sets, and uses the `-`:

Exercise

```
fruits_set - grocery_set
grocery_set - fruits_set
```

Question 7. Again, let `dairy_set = {'milk', 'cheese', 'cream'}`. What will `dairy_set - grocery_set` be?

☐ `None`
☐ `{'cream'}`
☐ `{'milk', 'cheese'}`
☐ `{'cream', 'cheese', 'apple', 'milk', 'egg'}`

Sets can have different data types! Just like lists!

Exercise

```
Random_set = {"California", 38, 3.14159} #A string, int, float
Random_set
```

`type()` Remember that we can call `type()` on an object, including sets, to see type it is!

Exercise

```
print(type(Random_set))
```

We can also find the size of a set:

Exercise

```
len(Random_set)
```

Accessing values from a set Sets are *unordered*. Therefore, elements of a set are not assigned any indices. This means that you cannot access an element in a set by indexing!

Operations and manipulation on sets We can:

- Insert items into sets!
- Remove items from sets!

How do we do this? By using the built-in set methods.

Adding to a set You can add an item to a set with the `.add()` method:

Exercise

```
grocery_set.add('bread')
grocery_set
```

Note we didn't assign the results to `grocery_set`. Just like lists, the output was automatically changed.

Your turn! Try adding `'butter'` to `grocery_set`.

Exercise — write your code here:

Removing from a set You can remove an item from a set with the `.discard()` method:

Exercise

```
grocery_set.discard('egg')
grocery_set
```

Subsets A set where all of the elements belong to a larger set is known as a subset. You can check to see if one set is a subset of another using the method `.issubset()`.

Exercise

```
toast_set = {'bread', 'butter'}
toast_set.issubset(grocery_set)

toast_set.add('cinnamon')
toast_set.add('sugar')
toast_set.issubset(grocery_set)
```

How do we find methods? Use the built-in function `dir()`.

Exercise

```
print(dir(grocery_set))
```

Let's play with sets! Use the following two sets for the exercises below:

```
my_set_1 = {'apple', 'banana', 'cherry', 'date'}
my_set_2 = {'elderberry', 'fig', 'grape', 'honeydew'}
```

1. Confirm that these sets have no elements in common, i.e., are *disjoint*. You may want to look up a new method!

Exercise — write your code here:

2. Create a new set that contains all of the elements in `my_set_1` and `my_set_2`. Call this set `my_big_set`.

Exercise — write your code here:

3. Confirm that `my_set_1` is a subset of `my_big_set`.

Exercise — write your code here:

4. Write a line of code to see if `'plum'` is in `my_big_set`.

Exercise — write your code here:

Tuples

Like lists, a **tuple** is an ordered sequence of values. However, unlike lists or sets, tuples cannot be changed (including adding and removing elements) after they are created–this is known as being **immutable**.

Tuples can be helpful when data are structured like Cartesian coordinates, but can also be generalized to a greater number of dimensions. These objects often can be stored together, but are somewhat nonsensical to store as a list.

Consider a list of universities with the following information:

- Name
- City
- State

It might make more sense to bundle this information as a tuple (also called "packing").

You can create a tuple by using parentheses (`()`). Each element in the tuple should be separated by commas.

Let's create a tuple! Let's create a tuple representing the University of Chicago and its city and state.

Exercise

```
uc = ("University of Chicago", "Chicago", "Illinois")
uc
```

Tuples can have different data types! Let's say that we want to add an element to the tuple for the University's undergraduate enrollment (which requires an integer type, not a string). Remember, you can't change a tuple after it has been made because it is immutable! This includes adding or removing elements. You would have to overwrite the tuple or create a new one.

Exercise

```
uc_enrollment = ("University of Chicago", "Chicago", "Illinois", 7653)
uc_enrollment
```

`type()` Remember that we can call `type()` on an object, including sets, to see what type it is!

Exercise

```
type(uc_enrollment)
```

We can also find the length of a tuple:

Exercise

```
len(uc_enrollment)
```

Accessing values from a tuple Because tuples are ordered, each element of the tuple is assigned an index corresponding to its position where **indices start at 0**. We can access an element by calling the tuple and putting in square brackets the element we want!

Exercise

```
uc_enrollment[1]
```

Just like lists, negative indexes count from the end of the tuple.

Exercise

```
uc_enrollment[-1]
```

9. What will be the result of running the following code? Answer in the box below.

```
stanford_enrollment = ("Stanford University", "Stanford", "California",
↪  7554)
stanford_enrollment[2]
```

Question 8. What will be the result of running the code above?
☐ `'Stanford University'`
☐ `'Stanford'`
☐ `'California'`
☐ `7554`

Tuple methods Python only has two built-in methods for tuples!

- `count()`: Returns the number of times a specified value occurs in a tuple.
- `index()`: Searches the tuple for a specified value and returns the position of where it was found

Write a line of code to count how many times `'Illinois'` occurs in `uc_enrollment`.

Exercise — write your code here:

Write a line of code to find which index represents `'Illinois'` in `uc_enrollment`.

Exercise — write your code here:

Nested sequences Create a new tuple, `northwestern_enrollment`, in the same form as `uc_enrollment`, knowing that Northwestern University is located in Evanston, Illinois, and has an undergraduate enrollment of 8,709.

Exercise — write your code here:

It would be useful to store our two college-related tuples together along with any we create in the future. Create a list containing these two tuples in alphabetical order by college name. Let's call the list `tuple_list`.

Exercise

```
tuple_list = ...
tuple_list
```

Dictionaries

A **dictionary** is a set of "key: value" pairs where each key is unique. We can create a dictionary with curly brackets "{}". Entries of a dictionary are of the form "key: value".

Exercise

```
survey_dict = {0: "Strongly Disagree", 1: "Disagree", 2: "No opinion",
 ↪  3: "Agree", 4: "Strongly Agree"}
```

We can extract values of a dictionary by their key–this is why all keys must be unique!

Exercise

```
survey_dict[1]
```

Dictionaries are useful for storing and extracting data! Think of them like an address book where you look up someone's address by finding their name.

A few useful operations with dictionaries:

- Add an entry
- Delete an entry

We can delete pairs:

Exercise

```
del survey_dict[1]
survey_dict
```

Or add new pairs:

Exercise

```
survey_dict['new_key'] = 'new value'
survey_dict
```

We can also determine if keys are contained in the dictionary

Exercise

```
3 in survey_dict
```

Exercise

```
"Disagree" in survey_dict
```

Exercise

```
'new_key' in survey_dict
```

Keep in mind, keys do not all need to be the same type...although it may make more sense to keep them that way.

Exercise

```
survey_dict_2 = {"Strongly Disagree": 0 , "Disagree": 1 , 2: "No
↪  opinion", 3: "Agree", 4: "Strongly Agree"}
```

Exercise

```
survey_dict_2["Disagree"]
```

Exercise

```
list(survey_dict_2)
```

Dictionary Methods

There are methods that you can use on dictionaries, just like with lists and arrays. Here are a few:

Exercise

```
print(dir(survey_dict_2))
```

Exercise

```
survey_dict_2.keys()
```

Exercise

```
survey_dict_2.items()
```

Working with Dictionaries

Below is a dictionary containing total number of homicides in the United States in 2021, by state. (Published by Statistal Research Department, Oct 14, 2022). Note Washington D.C. is included as 'District of Columbia'.

Exercise

```
homicide_dict = {'Texas': 2064, 'North Carolina': 928, 'Ohio': 824,
 ↪  'Michigan': 747, 'Georgia': 728, 'Tennessee': 672, 'Missouri':
 ↪  593, 'Virginia': 562, 'South Carolina': 548, 'Illinois': 514,
 ↪  'Pennsylvania': 510, 'Louisiana': 447, 'Indiana': 438, 'Alabama':
 ↪  370, 'Kentucky': 365, 'Colorado': 358, 'Washington': 325,
 ↪  'Arkansas': 321, 'Wisconsin': 315, 'Oklahoma': 284, 'Nevada': 232,
 ↪  'Minnesota': 203, 'Arizona': 190, 'Oregon': 188, 'New Mexico':
 ↪  169, 'Mississippi': 149, 'Connecticut': 148, 'Maryland': 138, 'New
 ↪  Jersey': 137, 'Massachusetts': 132, 'New York': 124, 'California':
 ↪  123, 'District of Columbia': 109, 'West Virginia': 95, 'Delaware':
 ↪  94, 'Kansas': 87, 'Utah': 85, 'Iowa': 70, 'Rhode Island': 38,
 ↪  'Idaho': 36, 'Montana': 31, 'South Dakota': 26, 'Nebraska': 25,
 ↪  'Alaska': 18, 'Maine': 18, 'Wyoming': 17, 'New Hampshire': 14,
 ↪  'North Dakota': 14, 'Vermont': 8, 'Hawaii': 6}
```

Pick a few states of interest and find their homicide number (use code here...do not just manually search the dictionary!)

Exercise — write your code here:

20. Find the number of keys in the dictionary.

Exercise — write your code here:

Discuss with a neighbor (or on Ed):

1. Does this imply all 50 states are included here?
2. What are some limitations to this data?

Arrays

Arrays (from the `NumPy` library) are another way to collect data. Like a list, they contain a sequence of values. But, unlike a list, all elements of an array must have the same data type. This is because `NumPy` arrays were built for efficient computation. They can perform operations on all elements in one step. They can also do element-wise computation. This means that, for example, if you add two arrays together, the result will be an array where the element at each index is the sum of the elements at the index in the two original arrays. For this reason, when two arrays are added (or subtracted, multiplied, or divided) they must have the same size.

Lists vs Arrays

Lists are more flexible - Can contain elements of different types

`NumPy` arrays have some advantages

- Size - they take up less computer memory than lists.
- Performance - faster access than lists.
- Functionality - linear algebra functions built-in.
- Can be multiple dimensions.

Creating an array

First import the `NumPy` library!! We do this using an import statement and we give `NumPy` the alias `np`. This makes it so that any time we want to access the `NumPy` library, we only need to type `np`.

We can create an array using `np.array()` which takes in a list of values of the same data type. Below is an example of writing the import statement and creating an empty array. The array is empty because the list has no elements.

```
import numpy as np

np.array([])
```

Now, let's actually import numpy and create an array with two elements.

Exercise

```
import numpy as np

np.array([1,2])
```

Below, we create a list that contains integers and a string.

Exercise

```
tomato_list = [22, 38, 26, 35, 35, 'tomato']
print(tomato_list)
```

Let's see what happens when we try to make an array from this list.

Exercise

```
tomato_array = np.array([22, 38, 26, 35, 35,'tomato'])
print(tomato_array)
```

A list can have different types, but an array will default to one. So you can see all `int`s were changed to `string`s (notice the single quotes around the numbers).

An `array` will make sure everything is the same type.

Exploring arrays

Let's create an array by converting a list to a `NumPy` array.

Exercise

```
prime50_array = np.array([2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,
 ↪  41, 43, 47])

print(type(prime50_array))
```

Extraction and slicing of one-dimensional arrays work exactly the same as lists!

10. Print the second element of `prime50_array`.

Exercise — write your code here:

11. Print the `array` starting at index 1 and up to (but excluding) index 2.

Exercise — write your code here:

12. Now consider:

```
prime50_array[1::2]

prime50_array[::2]
```

Question 9. Will the code above give the same or different outputs?
☐ Same
☐ Different

Arrays have attributes

Attributes are characteristics of an object. We can view an object's attributes by using the dot operator `.` similarly to when we used methods by using the syntax `object.method()`. However, to access an attribute, we don't use parentheses. (See https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html)

Two attributes of arrays are size, which we can access using `.size`, and shape, accessed using `.shape`.

Exercise

```
prime50_array.size
```

Exercise

```
prime50_array.shape
```

Arrays have useful methods

Like lists, there are a lot of useful methods we can use on array objects. But unlike lists, we can always sum the elements in an array, since they are all numbers.

See the numpy documentation for a complete list of methods.

Exercise

```
print(prime50_array.sum())
```

Numpy also has many useful built-in functions to use on arrays!

Notice there is often more than one way to do a common operation!

Exercise

```
print(prime50_array.sum()) #sum method

print(np.sum(prime50_array)) #sum function (from NumPy library)

print(np.count_nonzero(np.array([1,2,0,2,1,0,2])))

print(prime50_array.mean()) #mean method

print(np.mean(prime50_array)) #mean function
```

We can easily create arrays by specifying a range. Calling `np.arange()` creates a half-closed interval [start,end) - the end value is not included

Exercise

```
np.arange(4,10)
```

If you leave out the start, the default is zero.

Exercise

```
print(np.arange(10))
```

We can specify a step size we want to increment by. If we leave out the step, the default is one

Exercise

```
print(np.arange(1,31,2))
```

Another reason why arrays are useful!

Element-wise operations!

Take a moment to think about what the following will produce before running the cell.

Exercise

```
array_1 = np.arange(10)
array_2 = np.array([1,2,3,4,5,6,7,8,9,10])
difference_array = array_1 - array_2
difference_array
```

13. Consider the following code:

```
import numpy as np
a = np.array([1,2,3,5,8])
b = np.array([0,3,4,2,1])
c = a + b
c = c*a
print(c[2])
```

Question 10. What will be the result of the code above?
Answer:

_____

14. Consider the following code:

```
number_array = np.array([1,2,3,5,8])
number_array = number_array + 1
print(number_array[1])
```

Question 11. What will be the result of the code above?
Answer:

_____

We may have higher dimensional arrays.

These are organized into rows and columns and are built row-by-row.

Exercise

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6]]) #specify first row then
 ↪  second

print(arr_2d)
```

Exercise

```
arr_2d.shape #outputs (#rows, #columns)
```

We can reshape our arrays using `np.reshape()` which takes in the array to reshape as well as the new shape.

When the new shape provided is an integer, the array produced will be 1-dimensional.

Exercise

```
reshaped_arr = np.reshape(arr_2d, 6)

reshaped_arr.shape
```

When the new shape is a tuple, the array produced will have the same number of dimensions as elements in the tuple.

Exercise

```
np.reshape(arr_2d, (3,2))

np.reshape(arr_2d, (3,2)).shape
```

Sometimes arrays are large and we don't want to do the math to figure out the size of each dimension. If we know we want it to have 3 rows. We can use -1 for the second dimension. Using -1, tells Python to infer the remaining dimension from the size of the input array.

Exercise

```
np.reshape(arr_2d, (3,-1)) # the unspecified value is inferred to be 2

np.reshape(arr_2d, (3,-1)).shape
```

Exercise

```
arr_2d[:,2] #all of the rows, column at index 2
```

Exercise

```
arr_2d[0,1] #element with row index 0 and column index 1
```

Wrapping Up

In this activity, we learned about lists, `arrays`, dictionaries, sets, and tuples and were introduced to the concept of DataFrames. In future activities we will build on these concepts until you feel comfortable working with and manipulating large groups of data.

Data Frames

DATA 118 Lab — Student Version

Goals

- To continue practicing with data frames, including their attributes, methods, and `pandas` functions.
- To carry out basic exploratory data analysis with aggregation functions and `.groupby()`.
- To examine relationships between variables using `pivot_table()`.
- To join data frames using `.merge()`.

Introduction to DataFrames

One of the most commonly used collections of data in data science is the DataFrame. Let's do a quick introduction before we get into working with them.

Different objects for different goals

There are different collections of data in Python because each works well in different contexts. We've already talked about lists and arrays:

Method 1: Lists (built-in to Python)

- Ordered and indexed
- Powerful built-in methods for sequences of data
- Flexible data types

Method 2: Arrays (must import `numpy`)

- All values of same data type
- Optimized for mathematical and matrix manipulations
- No row or column names

DataFrames two-dimensional tabular data structures that are especially useful if we have multiple observations of data of different types, or if we'd like for the data to be labeled.

Method 3: Data frames (must import `pandas`)

- Different columns can have different types
- Easy to manipulate by name
- Row and column names are built in

- Rows are "instances" as a `Series` object
- Columns are attributes of those individuals

Setup

For this lab we will be using the `pandas` library in Python, as well as a new dataset containing information on various Star Wars characters from Episodes I-VII. We have several data frames to use as a part of the larger dataset–run the cell below to setup the environment.

Exercise

```
import pandas as pd

# To download any of these data files go to
# https://posit.ds.uchicago.edu/correlationlab/www/<name of file
 ↪  here>.csv

starwars = pd.read_csv("./data/starwars.csv")
```

Data frame attributes

First, let's review some basic `pandas` using the `starwars` data frame. `starwars` includes the following variables:

- `name`: Name of the character
- `height`: Height (cm)
- `mass`: Weight (kg)
- `hair_color`, `skin_color`, `eye_color`: Hair, skin, and eye colors
- `birth_year`: Year born (BBY = Before Battle of Yavin)
- `sex`: The biological sex of the character, namely male, female, hermaphroditic, or none (as is the case for Droids)
- `gender`: The gender role or gender identity of the character as determined by their personality or the way they were programmed (as is the case for Droids)
- `homeworld`: Name of homeworld
- `species`: Name of species

Question 1. Which data frame attribute can confirm the names of these columns?

☐ `.columns`
☐ `.dtypes`
☐ `.index`
☐ `.shape`
☐ `.size`

Question 2. Which data frame attribute can help you find the number of observations in `starwars`?

☐ `.columns`
☐ `.dtypes`
☐ `.index`
☐ `.shape`
☐ `.size`

Use the method you chose above on our data frame.

Exercise — write your code here:

Question 3. How many observations are there in `starwars`?

☐ 11
☐ 87
☐ 1044

Discuss with a neighbor (or on Ed):
In the exercise above we asked you to use an attribute to find the number of observations in the data frame. It is also possible to use a function we have seen before. Can you guess what it is? Try to use it and discuss the difference in output with your neighbors.

Now let's find out what the `.dtypes` attribute does:

Exercise

```
starwars.dtypes
```

Remember that you can always look at the documentation for a function, method, or attribute–including the documentation for `.dtypes`. Hopefully after reading, you have found that this attribute is a `Series` object that describes the data found in each column.

Question 4. Which of the variables in `starwars` are numeric? Select all that apply.

☐ `name`
☐ `height`
☐ `mass`
☐ `hair_color`
☐ `skin_color`
☐ `eye_color`
☐ `birth_year`
☐ `sex`
☐ `gender`
☐ `homeworld`
☐ `species`

Categorical variables are usually saved as strings, but all we can see is `object`. Let's investigate further. Using square brackets, extract the `name` column.

Important note: Throughout this lab, please use double quotes instead of single quotes when dealing with strings: e.g., "this" not 'this'.

Exercise — write your code here:

Now, apply the `type()` function to the extracted column.

Exercise — write your code here:

Because it's a column of the data frame, `name` is saved as a `Series` object! But, we can still confirm that the `Series` is made up of strings. Write a line of code that checks the type of the first element in the column. Extract the column first, and then the row.

Exercise — write your code here:

You should finally see that this is indeed a string.

Now, write a line of code using double square brackets to select all of the numeric variables. Keep them in the same order as the original data frame.

Exercise — write your code here:

Write a line of code to extract the eleventh row of the data frame. Specify that we are selecting all columns.

Exercise — write your code here:

Remember that when indexing with square brackets in Python, the first index is inclusive and the second is exclusive. Write a line of code using `iloc` to select the tenth through nineteenth rows of the data frame. For this question, do not specify that we are selecting every column; `pandas` does this by default.

Exercise — write your code here:

Don’t forget, we can also subset the data frame using row extraction in conjunction with a logical operator to select the observations that meet a certain criteria.

Question 5. Which statement below tells us if each observation is a Droid?

☐ `starwars["species"] = Droid`
☐ `starwars["species"] == Droid`
☐ `starwars["species"] = "Droid"`
☐ `starwars["species"] == "Droid"`

Now, using your answer as well as `.loc`, extract the rows of `starwars` where the character is a Droid.

Exercise — write your code here:

Finally, write a line of code that will sort the data frame according to the character's mass. Write your code so that the largest masses appear first.

Exercise — write your code here:

Question 6. Which character has the largest mass?

☐ Captain Phasma
☐ Jabba Desilijic Tiure
☐ Yarael Poof
☐ Yoda

`.groupby()`

Now let's do some exploratory data analysis with `.groupby()` and our aggregation functions. Recall that the basic syntax is: `df.groupby("column_name")["value_column"].func()`. We like to use the following functions:

- `.count()`: The number of observations
- `.mean()`: The average of the observations
- `.min()`: The smallest observation
- `.max()`: The largest observation

Write a line of code to find out how many of each species there are in the data frame. Since everyone likely is labeled with a name, let's use `name` as the `value_column`.

Exercise — write your code here:

Discuss with a neighbor (or on Ed):
Note that while we are asking you to use `.groupby()` to complete these problems, we can also `.value_counts()`. See if you can make it work, and discuss the difference in output with your neighbors.

Save your result as a new data frame, `starwars_species`. Right below it, write one new line of code to give us the number of species with more than one character in the *Star Wars* universe.

Exercise — write your code here:

Question 7. How many species are there with more than one character?
Answer:
_____

Using the original `starwars` data frame, write a single line of code to calculate the mean mass for each of the species. Normally we would not do this for very very small groups (e.g., one observation), but for now, it's okay.

Exercise — write your code here:

Save your result as a new data frame, `starwars_species_mass`. Then, write one line of code to sort the species that have an average mass of less than 100 kg from most to least massive.

Exercise — write your code here:

Pivot tables

Let's transition to working with pivot tables. Recall that when working with pivot tables we provide `DataFrame`, `index`, `columns`, `values`, and `aggfunc` arguments.

Write a line of code to count the number of characters with each eye and hair color combination. Eye color should be in the rows of your pivot table. Use "name" in the `values` argument.

Exercise — write your code here:

Question 8. How many characters have black eyes and no hair?
☐ 1.0
☐ 3.0
☐ 9.0
☐ `NaN`

This is what is known as a "sparse" table–lots of categories with no values, and only a few with actual numbers. Let's assign our pivot table to a data frame called `pivot_df`, and use a method called `.stack()` on it.

Exercise — write your code here:

This helped us a lot! It dropped all NaN values, and put it back in a tabular form.

Also take note of the warning given to us by Python. There isn't an issue with running the code now, but there will be in the future.

Discuss with a neighbor (or on Ed):
In the exercise above, we asked you to use `.stack()` on a pivot table to make it more readable. Could we have used `.groupby()` on the original `starwars` data frame instead? Would there be any difference in output?

Now let's take a look at a smaller, less sparse table.

Write a line of code to create a pivot table that compares mean heights grouped by sex and species. `species` should be in the rows of your table.

Exercise — write your code here:

Question 9. What is the average height of a female human (in cm)?
☐ `NaN`

`.merge()`

Finally, let's consider `.merge()`. We have two variables in the `pilots` data frame– a string containing all vehicles a character has driven, and a string containing all starships a character has driven.

Exercise

```
pilots = pd.read_csv("./data/pilots.csv", index_col = 0)
```

Write one line to confirm which columns `pilots` has in common with `starwars`. To do this, make both column `Series` into sets, and then take the intersection. Return to last week's tutorial if you forgot about sets!

Exercise — write your code here:

Question 10. Which column do `pilots` and `starwars` have in common?

☐ `height`
☐ `name`
☐ `vehicles`
☐ `starships`

Merge `pilots` and `starwars` into one data frame called `starwars_big` using `pd.merge()`. No need to add any additional arguments.

Exercise — write your code here:

If we take a look, there are quite a few cells in the data frame with missing values (`NaN`).

If a character has not driven any vehicles, they will have an empty string `""` in the `vehicles` column in the `pilots` data frame (same with `starships`). Write a line of code to take a subset of `pilots` where the characters are only included if they have driven at least one vehicle or at least one starship Save the subset as `pilots_small`.

Hint: Use the `pandas` function `notna()` in conjunction with a binary operator.

Exercise — write your code here:

Now, merge `pilots_small` and `starwars` in such a way that ONLY the characters who have driven a vehicle or starship remain in the dataset. Call this dataframe `starwars_pilots`.

Exercise — write your code here:

Thus far, we have only shown you how to use `merge()` as a function. However, we can also use it as a method, where it acts on one of the data frames explicitly. Take a look at this example:

Exercise

```
starwars_pilots_method = pilots_small.merge(starwars, how = "left")
```

Both data frames: `starwars_pilots` and `starwars_pilots_method` should be identical. Write one line of code to check this using `.equals()`.

Exercise — write your code here:

Now that we have learned how to join two data frames that have a column in common, let's take a look at how to add multiple data frames together when they all have the same columns. For example, we have seven smaller data frames containing all of the characters that appear in Episodes I-VII.

Exercise

```
ep1 = pd.read_csv("./data/the_phantom_menace.csv", index_col = 0)
ep2 = pd.read_csv("./data/attack_of_the_clones.csv", index_col = 0)
ep3 = pd.read_csv("./data/revenge_of_the_sith.csv", index_col = 0)
ep4 = pd.read_csv("./data/a_new_hope.csv", index_col = 0)
ep5 = pd.read_csv("./data/the_empire_strikes_back.csv", index_col = 0)
ep6 = pd.read_csv("./data/return_of_the_jedi.csv", index_col = 0)
ep7 = pd.read_csv("./data/the_force_awakens.csv", index_col = 0)
```

Similarly to strings and lists, we can concatenate data frames together. The function to do so is `concat()`. `concat()` accepts a list of data frames to be concatenated.

Concatenate the `ep1` and `ep2` data frames.

Exercise — write your code here:

You can also provide a longer list to concatenate more data frames simultaneously. Concatenate all seven of the smaller `epx` data frames.

Exercise — write your code here:

Functions and Control Flow

DATA 118 Lab — Student Version

Goals

- To become familiar working with basic functions and writing our own.
- To explore using functions as arguments to other functions.
- To implement control flow in our code, which alters the linear path of code execution through:
  - Conditional statements: `if, elif, else` blocks
  - Iteration: `for` and `while` loops

Introduction and motivation

Up until this point, we have been writing code that executes linearly: Python reads our code line by line, without skipping or repeating any.

In this lab, we explore how to write Python code that repeats certain blocks, or skips them when they are not necessary. You might be wondering why we might want to do this. Consider the following scenarios:

- I am a researcher who runs experiments and calculates field-specific summary statistics at the end of each experiment. I don't want to repeat the code for my statistic calculation every time an experiment ends.
- I am writing code to automate sending birthday texts. However, I only want this code to execute **if** I haven't already sent a birthday text that day.
- I would like to print every element of a (very long) list, each on a new line, without explicitly writing code for each print statement.

All of these problems can be solved either with functions, control statements, or a combination of the two!

Beyond saving our own time and sanity, writing code that uses functions and control statements (conditionals and iteration) is a good organizational practice. One important facet of writing code is to consider its complexity – we want our code to be easily understood by anyone who looks over it. Functions and control statements, alongside other good practices like informative variable naming and documenting our code with comments, help us produce code that is easy to hand off to another person (or our future selves).

Setup

Consider the following example on movie theater finances:

- Profit: defined as revenue (money from ticket sales) – cost (studio fees)
- Revenue: calculated from ticket price multiplied by the number of attendees
- Cost: $\$180$ studio fee plus four cents for every attendee

Mathematically, this means:

$$\text{profit} = \text{price} \times (\text{attendees}) - (180 + 0.04 \times \text{attendees})$$

So, if tickets cost \$5 and we have 120 attendees, we can calculate the profit. Translate the information above into Python code in the following chunk:

Exercise

```
revenue = ...
cost = ...
profit = ...
profit
```

Constants and variables

Notice that the information from our motivating example depends on many variables. Some will change with time, and other won't.

First let's consider our constants: the studio fee and cost per attendee. Even though these values are likely to stay fixed over time, it is good practice to define them using variables in our code. This is both for readibility and ease of change.

Check out the Python style guide that we use in DATA120. According to this guide, it is best practice to write named constants in all capital letters at the top of our file, so it's easy to find them and understand what they are when reading code. Also, we prefer to write decimal values with leading zeros: `0.8` is preferred to `.8`.

Let's do so below:

Exercise

```
STUDIO_FEE =
COST_PER_ATTENDEE =
```

On the other hand, ticket price and number of attendees are dynamic, and likely to change over time. Therefore, they are better suited as *inputs* to our code chunk. At this point, we should consider moving our code into a **function**. Let's do that in the next section.

Function basics

Because we have a code chunk that we'd like to repeat with different values, our profit calculation is a great use case for a function.

Let's rewrite our profit calculation as a function called `calculate_profit` that takes arguments `ticket_price` and `num_attendees`. Write your constants above the function.

Exercise

```
# CONSTANTS

# YOUR FUNCTION IS DEFINED BELOW

......................
    revenue = ticket_price * num_attendees
    cost = STUDIO_FEE + COST_PER_ATTENDEE * num_attendees
    profit = revenue - cost
    ........
```

Check to see that your function works as expected by calling it with `ticket_price = 5` and `num_attendees = 120` again.

Exercise — write your code here:

Assume that we know that for every `$1` discount, `40` more people will come. What is the profit if we set `ticket_price = 4`? Use your function to figure out the answer.

Exercise — write your code here:

Question 1. What is our new profit if we decrease the ticket price to $4?
☐ $415.20
☐ $295.20
☐ $619.80
☐ $453.60

Great! Now we have a working function that allows us to calculate the new profit as ticket price and number of attendees changes.

Docstrings

Remember that we should be documenting what our functions do using docstrings.

Question 2. Which of the following are true about docstrings? *(Select all that apply.)*
☐ We can access them later using `.__doc__`
☐ We can access them later using `.__docstring__`
☐ We enclose docstrings in triple quotes
☐ Docstrings should be placed above the `def func()` line
☐ Docstrings should be placed below the `def func()` line, and indented with the function body

Now go back and add a docstring to your function if it doesn't have one!

Beyond basic functions

Before we begin making our functions more complex, let's cover a few more function-related topics that will come in handy.

`lambda` functions

We might find it unnecessary to write entire functions for very short computations. For example, revenue is just ticket price times attendees. Writing out a full `def` block for this feels a little clunky:

```
def calculate_revenue(ticket_price, num_attendees):
    return ticket_price * num_attendees
```

Python lets us write small, one-line functions using the `lambda` keyword. The general syntax is: `lambda arguments: expression`.

Our arguments go before the colon, and the single expression after the colon is what the function returns. Notice that there is no function name or `return` keyword; the result of the expression is returned automatically. Here is the same revenue calculation written as a `lambda` function:

```
calculate_revenue = lambda ticket_price, num_attendees: ticket_price *
 ↪  num_attendees
```

Because `lambda` functions don't *need* a name, they are sometimes called **anonymous functions**. We assigned this one to the variable `calculate_revenue` so we could call it, but `lambda` functions are actually most useful when we pass them directly into another function without ever naming them. We'll see this in the next section.

Now, write a `lambda` function that you assign to a variable `calculate_cost` that takes `num_attendees` and returns the total cost: the studio fee plus the cost per attendee times the number of attendees. Use the named constants `STUDIO_FEE` and `COST_PER_ATTENDEE`.

Exercise

```
calculate_cost = ...
```

Recall that we assumed every `$1` discount from our `$5` ticket price brings in `40` more people, starting from `120` attendees. Write a `lambda` function called `estimate_attendees` that takes `ticket_price` and returns the expected number of attendees.

Exercise

```
estimate_attendees = lambda ticket_price: ...

estimate_attendees(4)
```

Question 3. Which of the following are true about `lambda` functions? *(Select all that apply.)*

☐ The value of the expression after the colon is returned automatically
☐ They must include the `return` keyword
☐ They can contain multiple lines of statements, just like a `def` function
☐ They must be assigned to a variable name before they can be used
☐ They can take more than one argument

Functions as objects

In Python, *everything* is an object: integers, strings, lists, data frames… and functions too! This means a function can be treated like any other value. We can check its type, assign it to a new variable, or store it in a list.

```
print(type(calculate_profit))

profit_function = calculate_profit
print(profit_function(5, 120))
```

```
<class 'function'>
415.2
```

Notice that when we wrote `profit_function = calculate_profit`, we did **not** use parentheses. There is an important difference here:

- `calculate_profit` refers to the function object itself
- `calculate_profit(5, 120)` *calls* the function, running the code and giving back the result

Question 4. What is stored in `x` after running `x = calculate_profit`?
☐ The number `415.2`
☐ The function `calculate_profit` itself, which we can call later with `x(5, 120)`
☐ Nothing, because this line raises an error since no arguments were given
☐ The string `'calculate_profit'`

Passing functions as arguments Because functions are objects, we can pass them as arguments to other functions.

For example, our theater manager might have different models for how many people will show up at a given price. Instead of hard-coding one model, which is inflexible, we can write a function that takes `attendance_func` as an argument:

```
def profit_at_price(ticket_price, attendance_func):
    """
    Calculate the profit at a given ticket price, using attendance_func
    to estimate the number of attendees at that price.
    """
    num_attendees = attendance_func(ticket_price)
    return calculate_profit(ticket_price, num_attendees)
```

Try running calling `profit_at_price` with `ticket_price = 4`, and use our function `estimate_attendees` as `attendance_func`.

Exercise — write your code here:

Now, let's say that on weekends, the theater is busier: `200` people come at `$5`, and each `$1` discount brings in `50` more. We don't need to write a new named function for this. We can pass a `lambda` directly:

```
profit_at_price(4, lambda ticket_price: 200 + 50 * (5 - ticket_price))
```

```
810.0
```

Discuss with a neighbor (or on Ed):
Many built-in Python functions also accept functions as arguments. For example, `max()` has an optional `key` argument: a function used to score each element. As a challenge, try to write a line of code to find which ticket price in a list will give the highest profit given our weekday model of estimating attendance.

Returning functions Just as functions can be passed *into* other functions, they can also be returned *from* functions. This lets us write a function that builds and hands back new, customized functions.

Earlier we wrote two attendance models by hand, one for weekdays and one for weekends. They have the same shape and only differ in their numbers. Instead, we can write a function that *creates* attendance models:

```
def make_attendance_model(base_attendees, base_price,
↪  attendees_per_dollar):
    """
    Return a function that estimates attendance at a given ticket price.
    """
    return lambda ticket_price: base_attendees + attendees_per_dollar *
    ↪  (base_price - ticket_price)
```

Now play around a bit with `make_attendance_model`. Try to use it to recreate the weekday and weekend models we used earlier.

Exercise — write your code here:

Question 5. True or false: Running `weekday_model = make_attendance_model(120, 5, 40)` and `weekend_model = make_attendance_model(200, 5, 50)` calculates the expected attendance on weekdays and weekends.
☐ True
☐ False

Conditional statements

As you've gone about creating functions, you may have noticed times where it would have been useful to introduce some logic to your code. To put it in words: "if x is true,

then execute *this* code, but if y is true, execute *that* code". To implement such logic, we use **conditional statements**.

As a quick refresher, conditional statements in Python look like this:

```
if condition:
    # runs if condition is True
elif other_condition:
    # runs if condition is False and other_condition is True
else:
    # runs if none of the conditions above are True
```

Python checks each condition from top to bottom and runs only the **first** block whose condition is `True`. The `elif` and `else` parts are optional, and we can have as many `elif` blocks as we like. Conditions are usually built from comparisons (`==`, `!=`, `<`, `>`, `<=`, `>=`), membership checks (`in`), and logical operators (`and`, `or`, `not`).

Wednesday discount day

Let's return to our theater. To bring in more customers midweek, the theater runs a special: **every Wednesday, tickets are only `$3`**. Weekends are busy, so tickets on Saturday and Sunday cost `$6`. Every other day, tickets are the regular `$5`. On top of that, weekends use our busier `weekend_model` for attendance, while weekdays use the `weekday_model`.

We'd like a function `daily_profit(day)` that answers questions like "if today is Wednesday, what is the ticket price, and how much profit do we make?"

Trying to write this all at once, from scratch, would be messy: we'd be juggling prices, attendance models, and the profit formula inside one tangled function. When writing functions, aim for bite size pieces! Writing a very large program is an intimidating and daunting task, but if you break it up into smaller pieces and figure out how to solve each of those pieces separately, it is much easier.

So let's break `daily_profit` into smaller questions:

1. What is the ticket price on a given day?
2. Which attendance model should we use on a given day?
3. Given those two pieces, what is the profit? (We already have `profit_at_price` for this!)

Step 1: ticket price First, let's write `get_ticket_price(day)`, which returns the ticket price for a given day. We've written the constants for the prices and the weekend days for you. It's your job to fill in the conditions and return values.

Exercise

```
DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
 ↪  "Saturday", "Sunday"]
WEEKEND = ["Saturday", "Sunday"]
REGULAR_PRICE = 5
WEDNESDAY_PRICE = 3
WEEKEND_PRICE = 6

def get_ticket_price(day):
    if day == ...:
        return ...
    elif ...:
        return ...
    else:
        return ...
```

Hint: For the `elif` condition, use `in` to check whether `day` is one of the days in the `WEEKEND` list. For example, `"Sunday" in ["Saturday", "Sunday"]` is `True`.

Question 6. Answer this question without running any code: what will `get_ticket_price("wednesday")` return?

☐ 3
☐ 5
☐ 6
☐ An error

You might be tempted to write a series of `if` statements instead of `elif` and `else`, but this is often bad practice. Let's see why below:

```
def get_ticket_price(day):
    if day == "Wednesday":
        ticket_price = WEDNESDAY_PRICE
    if day in WEEKEND:
        ticket_price = WEEKEND_PRICE
    else:
        ticket_price = REGULAR_PRICE
    return ticket_price
```

Question 7. Answer this question without running any code: what will `get_ticket_price("Wednesday")` return now?

☐ 3
☐ 5
☐ 6
☐ An error, because `ticket_price` is assigned more than once

Step 2: attendance model Next, write `get_attendance_model(day)`. It should return `weekend_model` on weekends and `weekday_model` otherwise. Notice that this function returns a *function*, just like we saw in the last section!

Exercise

```
def get_attendance_model(day):
    if ...:
        return ...
    return ...
```

Note that this code only works if we created the weekend and weekday models earlier!

Discuss with a neighbor (or on Ed):
We wrote our code chunk above without an `else` statement. Was that a mistake? Do we need one here?

Step 3: putting it together Now that each piece works on its own, building `daily_profit(day)` is much easier. Use `get_ticket_price`, `get_attendance_model`, and `profit_at_price` to complete the function.

Exercise

```
def daily_profit(day):
    """
    Calculate the theater's profit on a given day of the week.
    """
    ticket_price = ...
    attendance_func = ...
    return ...

daily_profit("Wednesday")
```

Notice how short and readable `daily_profit` is. Each line reads almost like a sentence, and if something goes wrong, we can test each smaller function on its own to find the problem.

Use the chunk below to calculate the profit on a few different days.

Exercise — write your code here:

Question 8. How much profit does the theater make on a Wednesday? How about on a Monday?
☐ $412.00 on Wednesday, $415.20 on Monday
☐ $453.60 on Wednesday, $415.20 on Monday
☐ $412.00 on Wednesday, $714.00 on Monday
☐ $600.00 on Wednesday, $600.00 on Monday

Iteration

Using conditional statements, we can calculate the profit for any single day. But what if the theater manager asks, "how much would we make in a **week**?" We could call `daily_profit` seven times and add the results by hand, but that's repetitive, error-prone, and won't scale if we're asked about a month or a year. Instead, we can use a **loop** to repeat code for us.

`for` loops

A `for` loop runs a block of code once for each element in a sequence:

```
for item in sequence:
    # this code runs once for each item
```

Each time we go through the loop, `item` is assigned the next value in `sequence`. Once we reach the end of `sequence`, Python moves on to the code after the loop.

A very common sequence to loop over is a range of numbers created with `np.arange()`. Recall that `np.arange(start, stop, step)` creates an array of numbers starting at `start`, going up by `step`, and stopping *before* `stop`.

```
print(np.arange(7))
print(np.arange(3, 6.5, 0.5))
```

```
[0 1 2 3 4 5 6]
[3.  3.5 4.  4.5 5.  5.5 6. ]
```

Since `np.arange(7)` gives us `0` through `6`, we can use these numbers as indices into our `DAYS` list. Here is a loop that prints the profit for each day of the week:

```
for i in np.arange(7):
    day = DAYS[i]
    print(day, daily_profit(day))
```

```
Monday 415.2
Tuesday 415.2
Wednesday 415.2
Thursday 415.2
Friday 415.2
Saturday 714.0
Sunday 714.0
```

Adding up a week of profit

Printing is nice, but we want a single total. To do this, we create a variable *before* the loop that starts at `0`, and add to it each time through the loop.

Complete the loop below so that `total` holds the theater's total profit for one week.

Exercise

```
total = 0

for i in np.arange(...):
    day = ...
    total = ...

total
```

Question 9. What would happen if we moved `total = 0` *inside* the body of the loop?
☐ Nothing, the result would be the same
☐ `total` would be reset to `0` every time through the loop, so it would only hold Sunday's profit at the end
☐ Python would raise an error
☐ `total` would be seven times larger

Looping with `enumerate()` In the loop above, we used `np.arange(7)` to get an index `i`, and then used `DAYS[i]` to look up the day. This works, but Python gives us a cleaner option: `enumerate()`. When we loop over `enumerate(DAYS)`, each step gives us *two* values at once: the index and the element at that index.

```
total = 0

for i, day in enumerate(DAYS):
    profit = daily_profit(day)
    total = total + profit
    print("Day", i + 1, "is", day, "with profit", profit)

total
```

```
Day 1 is Monday with profit 415.2
Day 2 is Tuesday with profit 415.2
Day 3 is Wednesday with profit 415.2
Day 4 is Thursday with profit 415.2
Day 5 is Friday with profit 415.2
Day 6 is Saturday with profit 714.0
Day 7 is Sunday with profit 714.0
3504.0
```

Notice that we no longer need `DAYS[i]` or to know that there are exactly 7 days; `enumerate()` handles both for us. The index `i` still starts at `0`, which is why we print `i + 1`. If we didn't need the index at all, we could simply write `for day in DAYS:`.

A quick note on `while` loops

A `for` loop is the right tool when we know ahead of time how many times to repeat (for example, 7 days). Sometimes, though, we want to keep repeating *until a condition is met*, and we don't know in advance how long that will take. For this, Python has `while` loops:

```
while condition:
    # this code runs again and again, as long as condition is True
```

For example, suppose the theater wants to save up `$10,000` for a new projector. How many days will that take?

```
PROJECTOR_COST = 10000

savings = 0
num_days = 0

while savings < PROJECTOR_COST:
    day = DAYS[num_days % 7]
    savings = savings + daily_profit(day)
    num_days = num_days + 1

print(num_days, savings)
```

```
21 10512.0
```

Be careful: the body of a `while` loop must eventually make the condition `False`. If we forgot the line `savings = savings + daily_profit(day)`, `savings` would stay at `0` forever, and the loop would never stop. This is called an **infinite loop**. In practice, you'll use `for` loops much more often, but `while` loops are handy when the number of repetitions isn't known ahead of time.

Debugging Your Code

DATA 118 Lab — Student Version

Goals

The goal of this lab is to learn about common error messages and practice debugging your code.

Setup

For this lab, we will be using the `numpy` and `math` modules.

Exercise

```
import numpy as np
import math
```

What is a bug?

Debugging is the process of identifying and fixing 'bugs' in your code. A **bug** is computer science term for an error. This is thought have been popularized from "The 1947 Moth Incident", which involved a literal bug.

From the Smithsonian National Museum of American History:

> In 1947, engineers working on the Mark II computer at Harvard University found a moth stuck in one of the components. They taped the insect in their logbook and labeled it "first actual case of bug being found." The words "bug" and "debug" soon became a standard part of the language of computer programmers. Among those working on the Mark II in 1947 was mathematician and computer programmer Grace Hopper, who later became a Navy rear admiral. This log book was probably not Hopper's, but she and the rest of the Mark II team helped popularize the use of the term computer bug and the related phrase "debug."

IBM/Harvard University (1947). Log Book With Computer Bug [Photograph]. Smithsonian National Museum of American History. https://americanhistory.si.edu/collections/object/nmah_334663

Bugs may either

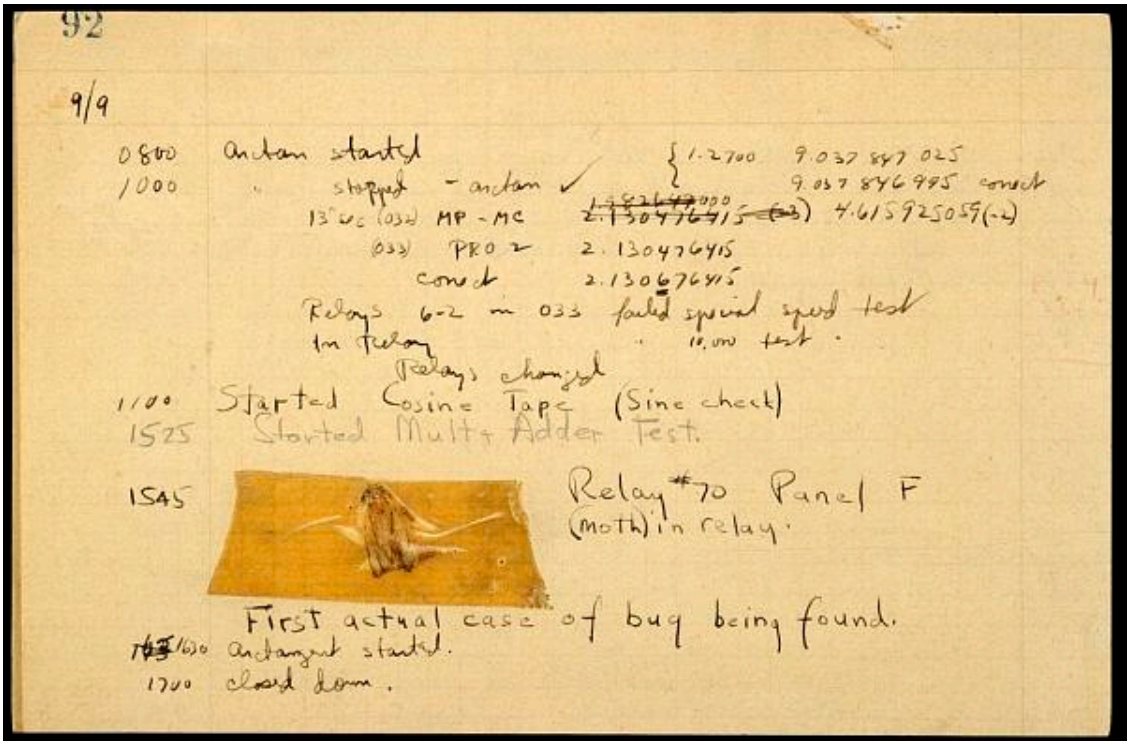


Figure 1: Log Book With Computer Bug [Photograph]

- Cause your code to produce an error message, or
- Cause your code to produce incorrect results (**logical errors**).

Error Messages

There are many specific types of error messages you might encounter when using Python that can generally be sorted into two main types:

- **Syntax Errors** - made before execution
  - `SyntaxError`
- **Exceptions** - made during execution
  - `NameError`
  - `IndexError`
  - `AttributeError`
  - `KeyError`
  - `ImportError`
  - `TypeError`
  - `ValueError`
  - `ModuleNotFoundError`
  - ...

When one of these errors occurs, Python will return an error message. We are very used to thinking of errors as a bad thing, but while programming, you may come to actually appreciate error messages, as they can help you go back and rewrite your code to run correctly.

Syntax Errors

If your code includes a syntax error, you will see a message starting with `SyntaxError`.

These messages typically include:

- A file name or cell input number,
- A line number, and
- An arrow (or arrows) pointing to the issue.

The issue could be that something is missing…

Exercise

```
for i in [1,2]
    print(i+8)
```

or that something was written incorrectly.

Exercise

```
for i in [1,2],
    print(i+8)
```

Sometimes, the software you are using will try to warn you about syntax errors ahead of time using red or yellow lines underneath problematic parts of your code. This does not show up in this lab setup, but if you copy and past the code below into an IDE like Jupyter or VSCode, you will see these lines.

Exercise

```
mean([1,2,3,4)
```

These errors occur any time we make a mistake with regards to the rules that define how Python code is written. For example, when Python sees a `for`-loop, it expects to see a colon `:`. When it sees an open bracket `[`, it expects to see a closed bracket `]`. If your syntax is not correct, Python does not execute the code. Once your code is free of syntax errors, it will be executed.

Sometimes you may get special types of `SyntaxErrors`. A common one is the `IndentationError`, which occurs when you don't have the proper indentation in a for-loop, function, etc

Exercise

```
for i in [1,2]:
print(i+1)
```

Other times, the `SyntaxErrors` can be very vague and unhelpful. It can be useful to use the line numbers to identify where the error is, even if you don't know exactly what the error is. Then, we recommend looking closely at the various "punctuation marks" to see what is missing or incorrect.

Exceptions

During code execution, you could receive other error messages known as exceptions. You will see a message starting with the exception type (e.g., a `NameError`).

Typically includes:

- A file name or cell input number,
- A line number,
- A brief explanation, and
- **traceback** - the context in which the exception occurred
  - Usually in the form of a few lines of code.
  - More common when you write functions.

The exception type should give you clues as to what went wrong.

Note:
Because we are working in a Shiny environment there is also a line that says "Run reticulate::py_last_error() for details." You may ignore this, it is not relevant to you.

Exercise

```
print(my_string)
```

Exercise

```
5 + "6"
```

Exercise

```
a = 1 + 2
c = a + b
```

Let's walk through some exception types and what they mean...

We've already seen `NameError` and `TypeError`.

- `NameErrors` occur when you try to use a variable name that hasn't been assigned yet.
  - This is very common when running cells in your Jupyter notebook out of order or when you've restarted your Kernel

- `TypeErrors` occur when you use the wrong data type for whatever function or operand you are using.
  - These are common when you don't know the datatype of a `DataFrame` column (i.e., `"2016"` looks like `2016`).
  - When addressing `TypeErrors`, it can be useful to use functions like `type()` or `dtype()`:
    * `type()` gives you the data type but, if used on a collection of data, will only give you the type of container (ie `array`, `Series`, ...).
    * `dtype()` gives you the data type of the *elements* inside the container

Exercise

```
print(type("2016"), type(np.array([1, 2, 3])), np.array([1, 2,
↪  3]).dtype)
```

Lookup Errors

`IndexErrors` and `KeyErrors` are both types of exceptions called `LookupErrors`. They occur when you try to access a dictionary with a key that is invalid, use an invalid index for an array or DataFrame, etc.

`IndexErrors` and `KeyErrors` commonly occur when:

- You use `.groupby()` or otherwise manipulate a Data Frame and forget you have a new index.
- You forget Python indexes start at `0`.
- You just mistype the key or index.

Exercise

```
my_dict = {"a":1, "b":2, "c":3}
my_dict[1]
```

Exercise

```
my_list = [1,2,3]
my_list[3]
```

Other Common Error Types

`AttributeErrors` occur when there has been a failure to set or access an attribute for some object. `AttributeErrors` commonnly occur when the object you're working with does not actually have that attribute–don't forget that attributes are characteristics specific to a particular type of object (like a list, or a data frame, etc.). They can also happen if you forget parentheses on a method–first, check to make sure you've

included all of the necessary characters, then, check to make sure your object is the type that you think it is.

Exercise

```
my_list.size
```

Exercise

```
my_list.mean
```

`ValueErrors` occur when an invalid value is given to a function or operand. These are common when using mathematical functions.

Exercise

```
import math
math.sqrt(-1)
```

Exercise

```
float("number")
```

These all come from base Python, but imported modules can have their own error types as well! You may encounter more as you learn about more modules and more programming techniques, but the error types we've covered should give you a pretty solid basis for debugging.

Logical Errors

Sometimes your code does not encounter an exception, but you don't get the output that you expected. This is very common, especially when writing your own functions! Unfortunately, Python won't return an error message here–it is up to you to test and examine your code until you are comfortable with the output it is producing.

Here is an example:

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = 10
    for number in my_array:
        if number < smallest:
            smallest = number
    return smallest

my_min(np.array([6,3,2,5]))
```

Exercise

```
my_min(np.array([23,47,13,78]))
```

This function isn't giving me the answer I would expect... I need to look at the code inside my function to see what is wrong.

```
smallest = 10
for number in my_array:
    if number < smallest:
        smallest = number
```

It can be useful to do a few things when trying to debug code:

1) Think through the logic without worrying about the code itself. Should this approach work?
   - Sometimes we write 'pseudocode' or draw diagrams
2) Add print statements to see what is happening throughout the code
   - Where are things changing?
   - It works as expected until which line?
   - If you are using conditionals, is the condition ever met?
   - If you are using loops, is the loop working the way you expect?

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = 10
    for number in my_array:
        print("The number is: ",number)
        print("The smallest so far is: ",smallest)
        if number < smallest:
            print("I made it into the if-statement")
            smallest = number
    return smallest

my_min(np.array([23,47,13,78]))
```

From my print statements, I can tell

- The loop is iterating over the array correctly,
- The smallest number is always 10, and
- The condition for my `if`-statement was never met.

10 is smaller than all of the numbers in my array so `smallest` never gets set to anything else! We should initialize `smallest` to be something much larger than 10.

Exercise

```
def my_min(my_array):
    '''A function that takes in an array of numbers and returns the
    ↪  smallest number'''
    smallest = math.inf
    for number in my_array:
        if number < smallest:
            smallest = number
    return smallest

my_min(np.array([23,47,13,78]))
```

Error Types Activity

Each of the code chunks below has code that would raise an error. Choose the name of the error that would be raised in the chunk. Discuss with your neighbors why these errors might appear! Then, select a possible fix.

For example,

```
15/0
```

The code in this chunk would raise a `ZeroDivision` error because you cannot divide

by zero. To fix it, you might replace the zero with some other integer or float value.

```
state_capitals = {"Illinois": "Springfield",
                  "Massachusetts": "Boston",
                  "Texas": "Austin",
                  "Virginia": "Richmond"}

state_capitals["Washington"]
```

Question 1. What kind of error will be raised in the code chunk above?

☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 2. How would you fix this error?
Answer:
_____

```
state_capitals["Virginia"]]
```

Question 3. What kind of error will be raised in the code chunk above?

☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 4. How would you fix this error?
☐ Remove the second close bracket (`]`).
☐ Add a second open bracket (`[`).
☐ Replace the brackets with parentheses (`()`), because `state_capitals` should be a function.
☐ Add a comma after `Virginia`, because we are indexing by rows.

```
states = 50
print("There are " + states + " in the US.")
```

Question 5. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 6. How would you fix this error?
☐ Convert `states` to a string using `str()`.
☐ Use an operator other than the plus sign (`+`) for correct string/integer concatenation.
☐ Rewrite the entire phrase as a string, there is no need to concatenate three separate pieces.
☐ Convert `There are` and `in the US` to be integers using `int()`.

```
capitals_data = {'Capital': ["Springfield", "Boston", "Austin",
 ↪  "Richmond"]}
capitals_df = pd.DataFrame.from_dict(capitals_data)
capitals_df.index = ["Illinois", "Massachusetts", "Texas", "Virginia"]

capitals_df.iloc[50,]
```

Question 7. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 8. How would you fix this error?
☐ Change the index (`50`) to one of `0`, `1`, `2`, or `3`.
☐ Change the index, `50`, to `49`, because Python uses zero-based indexing.
☐ Change `.iloc` to `.loc`.
☐ Use `orient = index` in `to_dict()`.

```
capitals_df.len()
```

Question 9. What kind of error will be raised in the code chunk above?
☐ `AttributeError`
☐ `IndexError`
☐ `KeyError`
☐ `NameError`
☐ `SyntaxError`
☐ `TypeError`
☐ `ValueError`
☐ `ZeroDivisionError`

Discuss with a neighbor (or on Ed):
Why does this code raise that kind of error?

Question 10. How would you fix this error?
☐ Use `capitals_df.shape`, because `.shape` is the attribute of the DataFrame that contains the dimensions.
☐ Use `capitals_df.shape()`, because `.shape()` is the method for a DataFrame that will calculate the dimensions.
☐ Use `len(capitals_df)`, because `len()` is a function and not a method.
☐ Use `capitals_df.length()`, because `.len()` is not the right method name.

Debugging Activity

Now, you try! Find and fix the syntax errors, exceptions, and logical errors in the code blocks below.

Debug this combination of shapes and colors. Recall that a standard indent in Python is 4 spaces.

Exercise

```
shapes = ["circle", "square", "rectangle", "triangle", "rhombus",]
colors = ["red", "green", "blue"]
for i in range(5):
for j in range(3):
    print("A group of " + colors[i] + " " + shape[j] + "s")
```

Debug this set of factorials.

Exercise

```
number = 5
 factorial = 1

for i in range(1, number);
    factorial = factorial * "i"

print(f"The factorial of {number} is {factorial")
```

Debug this sum of even numbers.

Exercise

```
numbers = [1 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_sum = 0

for nums in numbers
    if num % 2 == 0
        even_sum = even_sum + num
    else
        even_sum = even_sum - num

print(f"The sum of all even numbers in the list is: {even_sum}")
```

Practicing Probability

DATA 118 Lab — Student Version

Goals

Recall that in statistics and data science, the idea is to detect patterns in your variables to understand relationships, make predictions, and draw inferences. In order to be able to do so, we need to know how to talk about how variables theoretically behave using probability.

Information can be conveyed about probability in a few different ways. In class, we covered one-way tables, Venn diagrams, and two-way contingency tables–now, we will review those rules while also linking them back to topics we have already covered, including how to calculate probabilities using tools like lists, arrays, and data frames.

The goals of this lab are:

- To practice using the rules of probability, including the General Addition Rule, the Multiplication Rule, and the Complement Rule.
- To practice calculating conditional probabilities.
- To interpret various probability distribution displays, including Venn Diagrams, one-way tables, and contingency tables.
- To reinforce what we know about probability using other tools like data frames.

Setup

For this lab we will be using `math`, `numpy`, and `pandas`.

Exercise

```
import math
import numpy as np
import pandas as pd
```

One-Way Tables

Consider a fictitious university similar to the University of Chicago in many ways–however, a policy of this university is that students may choose one and only one major. We have a data frame with information from 500 of this university's students.

Exercise

```
# To download this file go to
 ↪  https://posit.ds.uchicago.edu/probabilitypracticelab/www/students.csv

student_majors = pd.read_csv("./data/students.csv")
```

Use this cell to carry out some exploratory data analysis using either built-in Python functions or `pandas`-specific methods and attributes. You will need to explore the `student_majors` data frame to answer the following questions.

Exercise — write your code here:

Question 1. How many students are there in the data frame? A.k.a., if we were to calculate probabilities for this data set, what will our denominator be?
Answer:
_____

Question 2. What kind of variable is `major`?
☐ Discrete (Numeric)
☐ Continuous (Numeric)
☐ Nominal (Categorical)

Question 3. Describe the sample space of student majors at this university.
☐ $S = \{0, 1, 2, ..., 497, 498, 499\}$
☐ $S = \{1, 2, 3, ..., 498, 499, 500\}$
☐ $S = \{$ `major`, `DATA118` `DATA119`, `DATA120` $\}$
☐ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC` $\}$
☐ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC`, `Other` $\}$

Now, use `.groupby()` on `student_majors` to find the counts of each major. Be sure to view on the `"SID"` column, and save your new object as `major_dist`.

Exercise — write your code here:

Now, use this cell, as well as your `major_dist` object, to calculate the probabilities described in the questions below.

Exercise — write your code here:

Question 4. What is the probability a student is a DATA major? Round to three decimal places.
Answer:
_____

Question 5. What is the probability a student is a DATA or an ECON major? Round to three decimal places.
Answer:
_____

Discuss with a neighbor (or on Ed):
How would your approach to answering the previous question change if this university allowed students to have more than one major?

Question 6. What is the complement of the event `major == 'Other'`?
□ $S = \{0, 1, 2, ..., 497, 498, 499\}$
□ $S = \{1, 2, 3, ..., 498, 499, 500\}$
□ $S = \{$ `major`, `DATA118` `DATA119`, `DATA120` $\}$
□ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC` $\}$
□ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC`, `Other` $\}$

Using `major_dist` and some comparison operators, confirm the Complement Rule using a student's major, a.k.a., confirm that the students with `major == "Other"` is equal to the number of students without a named major in the dataset.

Exercise — write your code here:

Hint
Don't forget to look up the Complement Rule if you need.

Hint
Did you type in raw numbers like 500, or did you use `sum()` on your `student_majors` o

Hint
Did you select rows in the data frame using .loc?

Hint
Did you use a comparison operator to find the rows with "Other" majors?

Hint
Did you use a different comparison operator to find the rows with anything but "Other"

Venn Diagrams

Now, consider the remaining three variables, `DATA118`, `DATA119`, and `DATA120`, which are all Booleans describing whether each of the students are enrolled in DATA118, 119, or 120. This data is visualized in the Venn Diagram below.

Now, use this cell, as well as the `student_majors` data frame and the Venn Diagram, to calculate the probabilities described in the questions below.

Exercise — write your code here:

Question 7. What is the probability that a student is enrolled in DATA118?
Answer:
_____

Question 8. What is the probability that a student is enrolled in DATA119?
Answer:
_____

Question 9. True or False: The event that a student is enrolled in DATA118 and the event that a student is enrolled in DATA119 are mutually exclusive.
☐ True
☐ False

If the event that a student is enrolled in DATA118 and the event that a student is enrolled in DATA119 were independent, what would the probability of a student taking DATA118 and DATA119 be? Use the cell below to calculate your response.

Exercise — write your code here:

Question 10. What is the probability of a student taking DATA118 and DATA119, assuming these two events are independent? Round to three decimal places.
Answer:
_____

Question 11. What is the probability of a student taking DATA118 and DATA119, according to the Venn Diagram? Round to three decimal places.
Answer:
_____

Discuss with a neighbor (or on Ed):
Are the answers to the two previous questions different enough to convince you that taking DATA118 and 119 are dependent? How different would they have to be before you started to believe there is some type of relationship between the two classes?

Question 12. According to the Venn Diagram, what is the probability that a student is taking at least one of DATA118, 119, and 120? Round to three decimal places.
Answer:
_____

Contingency Tables

A conditional probability, or the probability of event A occurring given that event B occurred, is defined as

$$P(A|B) = \frac{P(A \ \& \ B)}{P(B)}$$

Although we don't need a contingency table to calculate these kinds of probabilities, they are a helpful way to visualize different events. Recall that `.pivot_table()` is one way to create a contingency table.

Create a pivot table with majors in the rows of the pivot table and whether or not a student is taking DATA118 in the columns. Be sure to use the `"SID"` column as the `values`. You should be counting the students, not taking the means or using any other aggregating function.

Exercise — write your code here:

As it turns out, there are some ways to improve our pivot table. Check out the documentation for `pivot_table()`. Then, add the argument `margins = True` to your line from the previous exercise. Save this new pivot_table as `majors_x_118`.

Exercise — write your code here:

Discuss with a neighbor (or on Ed):
Between the documentation and the output of your last line of code, what does adding `margins = True` do to your pivot table?

Discuss with a neighbor (or on Ed):
Another very similar function that you could use to create a pivot table is `pd.crosstabs()`. Check out the documentation to see how you might be able

to use it.

Now let's focus on finding the probability that a student is taking DATA118 conditioned on their major. Use this cell, as well as `majors_x_118`, to calculate the probabilities described in the questions below. Remember that we can use counts to calculate conditional probabilities and that the denominators in $P(A \ \& \ B)$ and $P(B)$ will cancel each other out.

Exercise — write your code here:

Question 13. If you were to calculate the probability that a student is taking DATA118 given that they are a DATA major using the definition of conditional probability as defined above, what is event B?
☐ The event that the student is taking DATA118.
☐ The event that the student is not taking DATA118.
☐ The event that a student is a BIOS major.
☐ The event that a student is a DATA major.
☐ The event that a student is included in the data frame.

Question 14. What is the probability that a student is taking DATA118 given that they are a DATA major? Round to three decimal places.
Answer:
_____

Question 15. If you were to calculate the probability that a student is taking DATA118 given that they are a BIOS major using the definition of conditional probability as defined above, what is event A?
☐ The event that the student is taking DATA118.
☐ The event that the student is not taking DATA118.
☐ The event that a student is a BIOS major.
☐ The event that a student is a DATA major.
☐ The event that a student is included in the data frame.

Question 16. What is the probability that a student is taking DATA118 given that they are a BIOS major? Round to three decimal places.
Answer:
_____

Discuss with a neighbor (or on Ed):
Are $P($ `DATA118` $|$ `BIOS`$)$ and $P($ `DATA118` $|$ `DATA`$)$ the same or different? What do you think that says about the independence of the variables `major` and `DATA118`?

Use this cell, as well as `majors_x_118`, to calculate the probabilities described in the questions below.

Exercise — write your code here:

Question 17. If you were to calculate the probability that a student is a DATA major given that they are taking DATA118 using the definition of conditional probability as defined above, what is event B?
☐ The event that the student is taking DATA118.
☐ The event that the student is not taking DATA118.
☐ The event that a student is a BIOS major.
☐ The event that a student is a DATA major.
☐ The event that a student is included in the data frame.

Question 18. What is the probability that a student is a DATA major given that they are taking DATA118? Round to three decimal places.
Answer:
_____

Discuss with a neighbor (or on Ed):
Is $P($ `DATA` $|$ `DATA118` $)$ the same as $P($ `DATA118` $|$ `DATA` $)$? Should they be equal to each other?

Finally, use this cell, as well as your pivot table, to calculate some even more complicated probabilities described in the questions below.

Exercise — write your code here:

Question 19. What is the probability that a student is NOT a DATA major? Round to three decimal places.
Answer:
_____

Question 20. What is the probability that a student is NOT taking DATA118 given that they are NOT a DATA major? Round to three decimal places.
Answer:
_____

Question 21. What is the probability that a student is NOT a DATA major given that they are NOT taking DATA118? Round to three decimal places.
Answer:
_____

Introducing Numeric Variables

So far, we have mostly worked with categorical variables (e.g., a student's major, whether or not they are taking a particular class, etc.). However, we are often more interested in calculating probabilities associated with a **random variable**, where random variables are defined as a variable or process with a numerical outcome.

Create a new column in the `student_majors` data frame representing the number of Introductory Data Science courses (118, 119, 120) a student is taking at this university. Call the new column `num_classes`.

Exercise — write your code here:

Hint
```
## Don't forget that to find the number of introductory DATA courses a student at this uni
```

Hint
```
## You may want to look up the `.sum()` method for data frames.
```

Hint
```
## You looked up `.sum()`, but did you specify the axis correctly?
```

Hint
```
## You looked up `.sum()` and specified the axis correctly, but are you summing the corre
```

Question 22. Describe the sample space of the number of introductory DATA courses a student could be taking at this university.
□ $S = \{0, 1, 2, 3\}$
□ $S = \{0, 1, 2, ..., 497, 498, 499\}$
□ $S = \{$ `major`, `DATA118` `DATA119`, `DATA120` $\}$
□ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC` $\}$
□ $S = \{$ `BIOS`, `DATA`, `ECON`, `MATH`, `PLSC`, `Other` $\}$

Discuss with a neighbor (or on Ed):
You might have looked at the column to see what values are in the sample space, but can you use `set()` or `.unique()` to justify your response?

Now use `.groupby()` to find the distribution. Be sure to view on the `"SID"` column, and save your new object as `num_classes_dist`.

Exercise — write your code here:

Use this cell, as well as `num_classes_dist`, to calculate the probabilities described in the questions below.

Exercise — write your code here:

Question 23. What is the probability that a student is not taking any introductory DATA classes? Round to three decimal places.
Answer:
_____

Question 24. True or False: The event that a student is taking two introductory DATA courses and the event that a student is taking three introductory DATA courses are mutually exclusive.
☐ True
☐ False

Question 25. What is the probability that a student is taking two or three introductory DATA courses? Round to three decimal places.
Answer:
_____

Question 26. What is the probability that a student is taking at least one introductory DATA course? Round to three decimal places.
Answer:
_____

Discuss with a neighbor (or on Ed):
Think of at least two ways to calculate the answer from the previous question. It should be the same value as the one from the Venn Diagrams section!

Moving ahead, we will learn how to calculate characteristics like the expected value and variance of a numeric random variable.

Discuss with a neighbor (or on Ed):
What is the expected value of introductory DATA courses a student at our fictional university is taking? How did you arrive at this answer?

Probability Distributions

DATA 118 Lab — Student Version

Goals

In the Probability Practice lab, we calculated probabilities from tables of data. Often, though, we want to describe how a random variable behaves using a known probability distribution. In this lab, we will work with three of the most common ones: the Bernoulli, Binomial, and Normal distributions.

The goals of this lab are:

- To identify when the Bernoulli, Binomial, and Normal distributions are appropriate for a given situation.
- To calculate expected values and variances for a random variable using its distribution's parameters.
- To calculate probabilities for single values and for ranges of values, both by hand and with the PMF, PDF, and CDF methods in `scipy.stats`.
- To use the Normal distribution to compare observations from groups with different means and standard deviations.

Setup

For this lab we will be using `math`, `numpy`, `plotnine`, `pandas`, and `scipy.stats`.

Exercise

```
import math
import numpy as np
import pandas as pd
import plotnine as p9
import scipy
from scipy import stats
```

Probability Distributions

Often, we are interested in calculating probabilities associated with a **random variable**, which is a function that assigns a real number to each possible outcome of a random process. We usually represent random variables with a capital letter, such as X, Y, or Z.

To calculate a probability, we need to know the random variable's **probability distribution**, which described the probability of each possible outcome. The probabilities must satisfy the following rules:

1. The outcomes must be disjoint.
2. Each probability must be between 0 and 1.
3. The probabilities must total 1.

We already discussed simple probability distributions, which are often presented in a table. Beyond listing probabilites, we are more interested in looking at our variable and seeing if it fits a **known probability distribution**, like Bernoulli, Binomial, and Normal distributions. There are MANY more beyond these three; check out this interactive chart to explore further.

Regardless of which distribution we are talking about, the following rules generally apply:

- Specific distributions are generally appropriate in certain situations and not in others. It is your job to pick something that will work well!
- Distributions will generally be described by either a **P**robability **M**ass **F**unction, abbreviated as **PMF** (in the case of discrete variables) or a **P**robability **D**ensity **F**unction, abbreviated as **PDF** (in the case of continuous variables).
- Distributions will be characterized by parameters, like the mean (μ) and standard deviation (σ).
- Distributions almost always have an expected value and variance (if not, they are known as degenerate distributions). You can calculate the expected value and variance by hand using the formulas we learned in class, but sometimes there are shortcut formulas based on the parameters. You can look these formulas up to use them if the distribution is well known–it is often much easier to use them.

The `scipy.stats` module has a lot of tools for working with known distributions. Each distribution, like `stats.binom` or `stats.norm`, is an object. If you call it with the distribution's parameters, you get back a new object for that specific random variable, which you can save and reuse:

```
my_bernoulli_variable = stats.bernoulli(p = 0.5)
my_binomial_variable = stats.binom(n = 20, p = 0.5)
my_normal_variable = stats.norm(loc = 0, scale = 1)
```

Once you have defined your object, you can use the following methods:

- `.pmf()`/`.pdf()`: Returns probabilities calculated using the probability mass function (defined for discrete variables only) or the output of the probability density function (defined for continuous variables only) for a specific random variable.
- `.cdf()`: Returns the cumulative distribution function, which calculates probabilities for a range of values of a specific random variable.
- `.ppf()`: Returns the percent point function for a specific random variable, a.k.a., the percentiles of that random variable's distribution.
- `.rvs()`: Generates random variates of a specific random variable.

Here is what each method looks like for the objects we defined above:

```
# P(X = 10) for 20 trials with p = 0.5
my_binomial_variable.pmf(10)

# P(X <= 10)
my_binomial_variable.cdf(10)

# The 50th percentile (median) of the distribution
my_binomial_variable.ppf(0.5)

# Height of the standard Normal PDF at 0 (a density, not a probability)
my_normal_variable.pdf(0)

# P(Z <= 1.96)
my_normal_variable.cdf(1.96)

# The value with 97.5% of the distribution below it
my_normal_variable.ppf(0.975)

# Five random draws from a Bernoulli variable (random_state makes the draws
 ↪  the same every time)
my_bernoulli_variable.rvs(size = 5, random_state = 119)
```

```
np.float64(0.17619705200195307)
np.float64(0.5880985260009766)
np.float64(10.0)
np.float64(0.3989422804014327)
np.float64(0.9750021048517795)
np.float64(1.959963984540054)
array([1, 0, 0, 1, 1])
```

You can also skip saving the object and pass the parameters straight to the method. For example, `my_binomial_variable.pmf(3)` and `stats.binom.pmf(3, n = 20, p = 0.5)` give the same answer.

We will introduce you to the `bernoulli`, `binom`, and `norm` distribution objects, but `scipy.stats` has many more, including some mentioned in the above Probability Distribution Chart. You can learn more about them in the `scipy.stats` documentation. Take a look, choose your favorite, and remember its description. This might come up later...

Bernoulli Distribution

Bernoulli distributions are appropriate when the random variable in question can take one of two values: 1, with probability p, or 0, with probability $1 - p$. Because it takes only values 0 and 1, it is a discrete variable and is described with a probability mass function (PMF). Then, you can write the PMF as follows:

$$P(x) = \begin{cases} 1-p & \text{if } x = 0 \\ p & \text{if } x = 1 \\ 0 & \text{otherwise} \end{cases}$$

We can summarize the information about this distribution in a table, like the one shown below. Refer back to our notes on expected value and variance to calculate these values yourself.

Support	PMF	$E(X)$	$Var(X)$
$x \in \{0, 1\}$	$P(X = x) = \begin{cases} 1-p & \text{if } x = 0 \\ p & \text{if } x = 1 \end{cases}$	p	$p(1-p)$

The distribution object we use for a Bernoulli random variable can be created with `bernoulli`:

```
my_bernoulli_variable = stats.bernoulli(p = 0.5)
```

Note that you do have to supply the probability of success `p`. Now, let's practice some Bernoulli probability questions:

Suppose I work at a bakery that sells two types of pies: apple and pumpkin. The pies are boxed in such a way that I must open the box to identify what type of pie is inside. I know the probability that a randomly selected pie is an apple pie is 42%. Assume any samples are randomly selected from random boxes.

Question 1. What is a 'success', according to this problem?
☐ The pie inside is an apple pie.
☐ The pie inside is a pumpkin pie.

Use the cell below to calculate the probabilities described in the questions below.

Exercise — write your code here:

Question 2. What is the probability of a pumpkin pie?
Answer:
_____

Question 3. What is the probability of finding apple pies in three randomly selected boxes? Round to three decimal places.
Answer:
_____

Question 4. What is the probability of finding pumpkin pies in three randomly selected boxes? Round to three decimal places.
Answer:
_____

Notice that the Bernoulli distribution is not particularly interesting! So why are we covering it? Because it is a useful building block for many other distributions, like the Binomial.

Binomial Distribution

Binomial distributions are appropriate when the random variable is the number of successes in a sequence of n independent trials, where each trial can only be a success or failure. This type of outcome is often described as binary, and hopefully sounds familiar to you–a single trial can be described as a Bernoulli variable! To check if something can be described with a Binomial distribution, ask yourself:

1. Is the number of trials, n, fixed in advance?
2. Can each trial be described only as a success or failure, and is the probability of success constant throughout all n trials?
3. Is each trial independent (that is, the outcome of one trial is not affected by the outcome of another)?

If the answer to ALL of these questions is yes, then you have a Binomial random variable.

Because the random variable takes only values 0, 1, 2, 3, etc., it is a discrete variable and is described with a probability mass function (PMF). Deriving the expressions for the expected value and variance can be done by hand, but relies on some mathematical tricks–you can read about the derivation for the expected value of the Binomial Distribution if you are interested, but otherwise, we will just report the relevant equations:

Support	PMF	$E(X)$	$Var(X)$
$k \in \{0, 1, ..., n\}$	$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$

Here, $\binom{n}{k}$ is read as n choose k, where the choose function is defined as

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

The distribution object we use for a Binomial random variable can be created with `binom`:

```
my_binomial_variable = stats.binom(n = 20, p = 0.5)
```

Note that you do have to supply the number of trials `n` and the probability of success `p`. Now, let's practice some Binomial probability questions, with the same bakery from the previous section.

Let's say I have 27 boxes, and I want to figure out the probability that exactly 17 of them contain pumpkin pies. Let's consider a pumpkin pie to be a success this time.

Question 5. What is n in this problem?
Answer:
_____

Question 6. What is p in this problem?
Answer:
_____

Question 7. What is k in this problem?
Answer:
_____

Use the cell below with a `scipy.stats.binom` object to calculate the answers to the questions below. Thedocumentation for `scipy.stats.binom`, alongisde the examples in the Probability Distributions section of this lab should help you choose the correct methods.

Exercise — write your code here:

Question 8. What is the probability that 17 of 27 boxes contain pumpkin pies? Round to three decimal places.
Answer:
_____

Question 9. Of my 27 pies, how many should I expect to be apple? Round to two decimal places.
Answer:
_____

Question 10. Now assume I have three pies, like I did in the Bernoulli distribution section. What is the probability that all three are pumpkin? Round to three decimal places.
Answer:
_____

Question 11. What is the probability that all three are apple? Round to three decimal places.
Answer:
_____

Discuss with a neighbor:
Do your answers match the values in the previous section? Why or why not?

We may also want to calculate probabilities for a range of values. Because the values are mutually exclusive, we can either add probabilities together, or use another method described in the documentation for `scipy.stats.binom`.

Question 12. Find the probability that there are two or fewer apple pies out of three. Round to three decimal places.
Answer:
_____

Now, in the cell below, use code to check that using the method `.cdf()` is equivalent to adding up multiple individual probabilites using `.pmf()`.

Because computers store decimals with tiny rounding errors, compare your two outputs with `np.isclose()` rather than `==`.

Exercise — write your code here:

Now use the cell below to answer the problems that follow.

Exercise — write your code here:

Question 13. Now assume I have 32 pies. What is the probability that more than 23 are apple? Round to four decimal places.
Answer:
_____

Question 14. What is the probability that less than 8 are apple? Round to three decimal places.
Answer:
_____

Question 15. What is the probability that between 13 and 18 are apple? Treat 13 and 18 inclusively, that is, include them in the range you are looking for. Round to three decimal places.
Answer:
_____

Normal Distribution

The Normal distribution is unlike any of the other distributions we have learned so far, since it is **continuous**. Furthermore, it is characterized by two parameters: μ, the expected value or mean (sometimes referred to as the location parameter), and σ, the standard deviation (sometimes referred to as the scale parameter). There are many distributions that are well approximated by the Normal distribution, and in fact, it is the most important distribution to many statisticians.

Because a normally distributed variable is a continuous variable, it is described with a probability density function (PDF). Here are the relevant equations:

Support	PDF	$E(X)$	$Var(X)$
$x \in \mathbb{R}$	$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	μ	σ^2

Two notes:

- Like any continuous distribution, the probability of observing a single value is exactly and always 0. We are typically looking at finding the probability of a range of values, e.g., $P(X < 0)$, $P(X > 3)$, $P(-1 < X < 1)$.
- To find probabilities associated with a range of values, you need to *integrate* the PDF over a certain range (remember integrals from calculus?). But the PDF for the normal distribution cannot be integrated by hand! Instead, you need to use the CDF (**C**umulative **D**istribution **F**unction), which you can find in the documentation for `scipy.stats.norm`.

The distribution object we use for a Normal random variable can be created with `norm`:

```
my_normal_variable = stats.norm(loc = 0, scale = 1)
```

Note that you do have to supply the mean μ using the argument `loc` (for location) and the standard deviation σ using the argument `scale`. Now, let's practice some Normal distribution questions:

In triathlons, it is common for racers to be placed into age and gender groups. Friends Leo and Mary both completed the Hermosa Beach Triathlon, where Leo competed in the Men, Ages 30 – 34 group while Mary competed in the Women, Ages 25 – 29 group. Leo completed the race in 1:22:28 (4948 seconds), while Mary completed the race in 1:31:53 (5513 seconds). Leo finished faster, but they are curious about how they did within their respective groups. Can you help them? Here is some information on the performance of their groups:

- The finishing times of the Men, Ages 30 - 34 group has a mean of 4313 seconds with a standard deviation of 583 seconds.
- The finishing times of the Women, Ages 25 - 29 group has a mean of 5261 seconds with a standard deviation of 807 seconds.
- The distributions of finishing times for both groups are approximately Normal. Remember: a better performance corresponds to a faster finish.

Question 16. Write the distribution for the finishing triathlon times for Men Ages 30-34 using proper notation.
□ $N(0, 1)$
□ $N(1, 0)$
□ $N(4313, 583)$
□ $N(583, 4313)$

Question 17. Write the distribution for the finishing triathlon times for Women Ages 25-29 using proper notation.
□ $N(0, 1)$
□ $N(1, 0)$
□ $N(5261, 807)$
□ $N(807, 5261)$

Use the cell below and a `scipy.stats.norm` object to calculate the answers to the

questions below.

Exercise — write your code here:

Question 18. What percent of the triathletes did Leo finish faster than in his group? Give your answer as a percent, rounded to one decimal place.
Answer:
_____

Question 19. What percent of the triathletes did Mary finish faster than in her group? Give your answer as a percent, rounded to one decimal place.
Answer:
_____

Discuss with a neighbor:
Who did better in the triathlon, relative to their sex/age group? How do you know?

More Practice Questions

When practicing these questions, make sure you are asking yourself what distribution you should be using!

Cake Batter

The yield of a box of cake mix is normally distributed with a mean of 4.75 cups of batter and a standard deviation of 0.25 cups.

Question 20. Write this distribution in mathematical notation.
□ $N(4.75, 0.25)$
□ $N(0.25, 4.75)$
□ $N(0, 1)$
□ $N(1, 0)$

Use the cell below to calculate the answers to the questions below.

Exercise — write your code here:

Question 21. What is the probability that a box of cake mix yields less than 4.15 cups of batter? Round to three decimal places.
Answer:
_____

Question 22. What is the probability that the box of cake mix yielded more than 5.2 cups of batter? Round to three decimal places.
Answer:
_____

Question 23. What is the probability that the box of cake mix yields between 4.25 and 4.85 cups of batter? Round to three decimal places.
Answer:
_____

Coffee Drinkers

A 2017 study shows that 64% of Americans drink coffee every day.

Use the cell below to calculate the answers to the questions below.

Exercise — write your code here:

Question 24. In a group of 14 people, how many are expected to drink coffee every day? Round to two decimal places.
Answer:
_____

Question 25. In the same group of 14 Americans, what is the probability that exactly 9 drink coffee every day? Round to three decimal places.
Answer:
_____

Question 26. What is the probability that at least ten drink coffee every day? Round to three decimal places.
Answer:
_____

Question 27. What is the probability that at most 7 drink coffee every day? Round to three decimal places.
Answer:
_____

Question 28. What is the probability that between 3 and 8 (inclusive) drink coffee every day? Round to three decimal places.
Answer:
_____

Bookworms

One in four Americans did not read a single book last year .

Use the cell below to calculate the answers to the questions below.

Exercise — write your code here:

Question 29. What is the probability, p, that a randomly chosen American did not read a single book last year? Round to two decimal places.
Answer:
_____

Question 30. In a group of 24 Americans, how many are expected to have not read a single book last year?
Answer:
_____

Question 31. In a group of 10 people, what is the probability that exactly three did not read a single book last year? Round to three decimal places.
Answer:
_____

Question 32. In a group of 19 people, what is the probability that no more than 3 people did not read a single book last year? Round to three decimal places.
Answer:
_____